

DiSCo: Device-Server Collaborative LLM-Based Text Streaming Services

Ting Sun¹, Penghan Wang², Fan Lai^{1*}

¹ University of Illinois Urbana-Champaign, United States

² Purdue University, United States

suntcrick@gmail.com, wangpenghan381@gmail.com, fanlai@illinois.edu

Abstract

The rapid rise of large language models (LLMs) in text streaming services has introduced significant cost and Quality of Experience (QoE) challenges in serving millions of daily requests, especially in meeting Time-To-First-Token (TTFT) and Time-Between-Token (TBT) requirements for real-time interactions. Our real-world measurements show that both server-based and on-device deployments struggle to meet diverse QoE demands: server deployments face high costs and last-hop issues (e.g., Internet latency and dynamics), while on-device LLM inference is constrained by resources.

We introduce *DiSCo*, a device-server cooperative scheduler designed to optimize users' QoE by adaptively routing requests and migrating response generation between endpoints while maintaining cost constraints. *DiSCo* employs cost-aware scheduling, leveraging the predictable speed of on-device LLM inference with the flexible capacity of server-based inference to dispatch requests on the fly, while introducing a token-level migration mechanism to ensure consistent token delivery during migration. Evaluations on real-world workloads—including commercial services like OpenAI GPT and DeepSeek, and open-source deployments such as LLaMA3—show that *DiSCo* can improve users' QoE by reducing tail TTFT (11-52%) and mean TTFT (6-78%) across different model-device configurations, while dramatically reducing serving costs by up to 84% through its migration mechanism while maintaining comparable QoE levels.

1 Introduction

Large language models (LLMs) have revolutionized various applications, with over 60% focusing on conversational interactions such as chatbots (Grand View Research, 2023). Meeting high serving demands requires scaling deployments across

on-premise servers in the cloud and on-device inference, as seen in Apple Intelligence (Gunter et al., 2024) and Google's Gemini Nano (Google, 2024). The Quality of Experience (QoE) for interactive applications is primarily evaluated by two critical metrics: Time-To-First-Token (TTFT) in the prefill stage, which quantifies the initial response latency, and Time-Between-Token (TBT) during the decode stage, which measures the consistency of token delivery speed (Databricks, 2023; Liu et al., 2024a,c).

On-server deployments lower serving costs by sharing infrastructure among many requests but often introduce unpredictable high latency due to request queuing delays (Agrawal et al., 2024) and the internet speed fluctuations. While on-device deployment is able to serve increasingly capable LLMs with sufficient accuracy, it suffers from slow processing speeds for long prompts and high energy consumption. For example, a fully-charged iPhone can only operate for less than two hours running an LLM with 7B parameters (Liu et al., 2024d).

This paper introduces a novel paradigm for cost-constrained device-server cooperative inference. We incorporate both server usage (e.g., monetary costs) and device energy costs via a dynamic exchange rate, which can be adjusted by endpoint users to balance response generation between the cloud and devices. As such, we can strategically distribute inference requests between endpoints and dynamically migrate ongoing token generation to maximize QoE. However, realizing this vision presents several fundamental challenges:

- **Unified Cost Management:** Total serving costs include resource expenditures from both endpoints—monetary costs from server API usage and energy costs from device computation. The relative value of energy costs varies dynamically based on device context (e.g., battery level, charging status) and user preferences for server spending, making it challenging to establish a unified optimization strategy.

: 基于设备 - 服务器协作的大型语言模型文本流服务

孙廷, 王鹏涵, 樊来 美国伊利诺伊大学香槟分校 美国普渡大学

摘要

大型语言模型 (LLM) 在文本流服务中的快速兴起，在为每日数百万次请求提供服务时引入了显著的成本和用户体验 (QoE) 挑战，尤其是在满足实时交互的首次令牌时间 (TTFT) 和令牌间时间 (TBT) 要求方面。我们的实际测量显示，无论是基于服务器还是设备端的部署，都难以满足多样化的用户体验需求：服务器部署面临高成本和最后一跳问题（例如，互联网延迟和动态变化），而设备端的 LLM 推理则受限资源。

我们介绍了 *DiSCo*，一种设备 - 服务器协作调度器，旨在通过自适应路由请求并在端点之间迁移响应生成来优化用户的体验质量 (QoE)，同时保持成本约束。采用成本感知调度，利用设备上大型语言模型推理的可预测速度与基于服务器推理的灵活容量，实时调度请求，同时引入令牌级迁移机制，以确保在迁移过程中令牌的一致交付。在真实工作负载上的评估——包括像 OpenAI GPT 和 DeepSeek 这样的商业服务，以及像 LLaMA3 这样的开源部署——显示，可以通过降低尾部首次响应时间 (TTFT) 11%-52% 和平均 TTFT 6%-78%，在不同模型 - 设备配置下提高用户的 QoE，同时通过其迁移机制显著降低服务成本最多达 84%，同时保持可比较的 QoE 水平。

1 引言

大型语言模型 (LLM) 已经彻底改变了各种应用，其中超过 60% 的应用集中在对话交互，例如聊天机器人（，）。为了满足高服务需求，需要在

在云端的本地服务器和设备端推理，如苹果智能（Gunter 等，）和谷歌的 Gemini Nano（，）。交互式应用的体验质量 (QoE) 主要通过两个关键指标进行评估：预测阶段的首个标记时间 (TTFT)，用于量化初始响应延迟，以及解码阶段的标记间隔时间 (TBT)，用于衡量标记传输速度的一致性（，；Liu 等，，）。

服务器端部署通过在多个请求之间共享基础设施来降低服务成本，但经常由于请求排队延迟（Agrawal 等，）以及互联网速度波动而引入不可预测的高延迟。虽然设备端部署能够以足够的准确性提供越来越强大的大型语言模型，但在处理长提示时速度慢且能耗高。例如，一部充满电的 iPhone 在运行具有 7B 参数的 LLM 时，最多只能运作不到两小时（Liu 等，）。

本文介绍了一种用于成本受限的设备 - 服务器协同推理的新范式。我们通过动态交换率将服务器使用（例如，货币成本）和设备能耗成本结合起来，终端用户可以调整该交换率，以在云端和设备之间平衡响应生成。因此，我们可以在终端之间战略性地分配推理请求，并动态迁移正在进行的令牌生成以最大化用户体验。然而，实现这一愿景面临若干基本挑战：

统一成本管理：总体服务成本包括来自两个端点的资源支出——服务器 API 使用的货币成本和设备计算的能耗成本。能耗成本的相对价值会根据设备环境（例如，电池电量、充电状态）以及用户对服务器支出的偏好动态变化，这使得制定统一的优化策略具有挑战性。

* Corresponding author

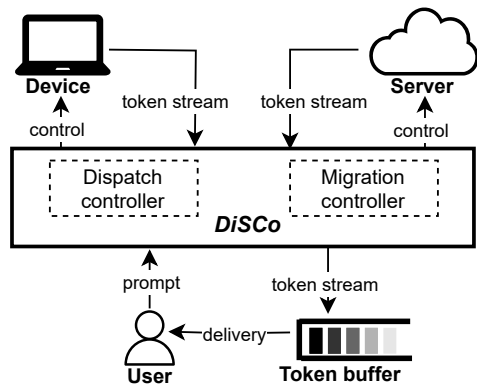


Figure 1: *DiSCo* acts as a middleware to optimize QoE by adaptively dispatching and migrating response generation between device and server endpoints under cost constraints.

- **Runtime Uncertainty:** The dynamic nature of networks (e.g., latency jitters) and serving loads make it challenging to accurately predict TTFT for in-flight request migration. Moreover, any scheduling mechanism must be lightweight to avoid introducing large overhead to the already latency-sensitive services.
- **Migration Impact on Token Delivery:** While dynamic migration between endpoints can reduce overall operating costs, it risks disrupting TBT. The challenge lies in determining when and how to migrate while minimizing user experience degradation and cost increase.

As shown in Figure 1, we introduce *DiSCo*, a Device-Server Cooperative scheduler that addresses these challenges via two key innovations:

- **Cost-Aware Dispatching Policies:** We introduce two dispatching mechanisms targeting different cost constraints. For server cost constraints, we employ a length-threshold based dispatching mechanism that routes requests shorter than a dynamically computed threshold to devices. For device energy constraints, we implement a delay-based dispatching mechanism where devices wait for a computed interval before starting local inference. Both mechanisms adapt their thresholds based on unified cost measures that combine server monetary costs and device energy consumption.
- **Token-Level Migration Framework:** We enable seamless generation handoff between endpoints through a novel migration protocol that preserves the consistency of token delivery. Our framework employs delayed migration timing to minimize interruption, while a token

buffer ensures smooth delivery during transitions. This design maintains user experience while saving resource costs across endpoints.

Through extensive evaluation using real-world traces from commercial LLM streaming API services, including GPT and DeepSeek, and on-device deployments, we demonstrate that *DiSCo* improves mean and tail TTFT by up to 50% without TBT violation, significantly reducing costs.

Overall, we make the following contributions:

- We characterize QoE challenges in device-server cooperative LLM inference through extensive real-world measurements.
- We design novel scheduling policies that optimize QoE under cost constraints.
- We develop a token-level migration framework to enable generation handoff between endpoints, preserving token delivery consistency.
- We demonstrate *DiSCo*’s effectiveness in commercial services and open-source benchmarks.

2 Background and Motivation

2.1 LLM Token Mixture and Routing

Device-server collaborative approaches have evolved in two directions. First, systems like EdgeShard (Zhang et al., 2024) and WDMoE (Xue et al., 2024a) partition LLMs across multiple endpoints when a single device cannot host the entire model. LLMcad (Xu et al., 2023) uses on-device models to reduce server costs, while PerLLM (Yang et al., 2024) optimizes energy consumption across devices and servers under constraints. Second, routing-based approaches (Ong et al., 2024; Ding et al., 2024) balance cost and accuracy by directing simple requests to small models and complex queries to advanced ones. However, these approaches do not optimize token delivery metrics (TTFT and TBT) under cost constraints.

2.2 LLM-Based Text Streaming Applications

Over 60% of LLM-backed applications focus on streaming conversational interactions, such as chatbots, virtual assistants, and language translation. QoE in these text streaming services is often quantified by two critical metrics: time-to-first-token (TTFT) for *initial responsiveness* and time-between-tokens (TBT) for *delivery smoothness* throughout the entire interaction timeline.

Current LLM systems struggle to meet user expectations for these metrics, with TTFTs ranging

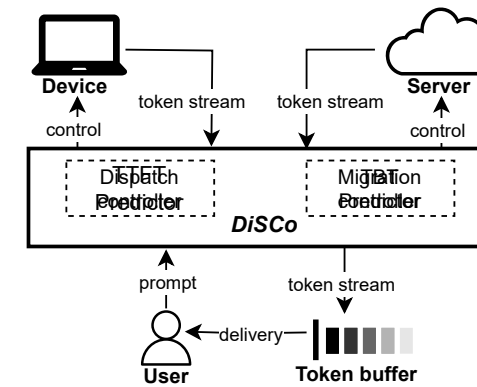


图 1: 作为中间件，通过在设备和服务器端之间自适应地调度和迁移响应生成，在成本约束下优化用户体验 (QoE)。

运行时不确定性：网络的动态特性（例如延迟抖动）和服务负载使得准确预测飞行中请求迁移的首次传输时间（TTFT）变得具有挑战性。此外，任何调度机制都必须轻量化，以避免给本已对延迟敏感的服务引入过大的开销。

迁移对令牌传递的影响：虽然在终端之间进行动态迁移可以降低整体运营成本，但它有可能破坏 TBT。挑战在于确定何时以及如何迁移，同时尽量减少用户体验的下降和成本的增加。

如图所示，我们介绍了，一种设备 - 服务器协作调度器，通过两个关键创新解决这些挑战：

成本感知调度策略：我们引入了两种针对不同成本约束的调度机制。对于服务器成本约束，我们采用基于长度阈值的调度机制，将长度低于动态计算阈值的请求路由到设备。对于设备能耗约束，我们实现了一种基于延迟的调度机制，设备在开始本地推理前会等待一个计算出的时间间隔。这两种机制都根据统一的成本度量调整其阈值，该度量结合了服务器货币成本和设备能耗。

令牌级迁移框架：我们通过一种新的迁移协议实现端点之间的无缝生成交接，该协议保持令牌传递的一致性。我们的框架采用延迟迁移时机以最小化中断，同时一个令牌

缓冲区确保在过渡期间平稳传输。这种设计在节省各端点资源成本的同时保持用户体验。

通过使用来自商业大语言模型流式 API 服务（包括 GPT 和 DeepSeek）以及设备端部署的真实数据进行广泛评估，我们证明了在不违反 TBT 的情况下，可将平均和尾部 TTFT 提高多达 50%，显著降低成本。

总体而言，我们做出了以下贡献：

我们通过大量的真实世界测量来描述设备 - 服务器协作的大型语言模型推理中的用户体验 (QoE) 挑战。

我们设计了在成本约束下优化用户体验的创新调度策略。

我们开发了一个基于令牌的迁移框架，以实现端点之间的生成交接，同时保持令牌传递的一致性。

我们展示了 在商业服务和开源基准中的有效性。

2 背景与动机

2.1 LLMTokenMixture 与路由

设备-服务器协作方法已经发展出了两个方向。首先，像 Edge-Shard (Zhang 等) 和 WDMoE (Xue 等) 这样的系统在单个设备无法承载整个模型时，将大语言模型分布到多个端点。LLMCad (Xu 等) 使用设备端模型来降低服务器成本，而 PerLLM (Yang 等) 在约束条件下优化设备和服务器的能耗。其次，基于路由的方法 (Ong 等 Ding 等) 通过将简单请求导向小模型，将复杂查询导向高级模型来平衡成本和准确性。然而，这些方法并未在成本约束下优化令牌传输指标 (TTFT 和 TBT)。

2.2 基于大型语言模型的文本流应用

超过 60% 的大型语言模型支持的应用程序专注于流式对话交互，例如聊天机器人、虚拟助理和语言翻译。这些文本流服务中的用户体验（QoE）通常通过两个关键指标来量化：首次响应时间（TTFT）用于衡量初始响应能力，以及整个交互过程中令牌间时间（TBT）用于衡量传输的平滑性。

目前的大型语言模型系统难以满足用户对这些指标的期望, TTFT 范围为

from hundreds of milliseconds to over ten seconds—far exceeding the ideal latencies of tens of milliseconds for interactive applications (Mäki-Patola and Härmäläinen, 2004; Žádník et al., 2022). Token consumption patterns vary by output modality: In visual text scenarios, reading speeds differ across demographic groups, with the majority (52%) aged 25-44 reading 4-5 tokens per second, while older groups generally read more slowly (Liu et al., 2024a; Brysbaert, 2019; Petrov et al., 2024). Audio output consumption shows more consistency, averaging 3-4 tokens per second across languages (Liu et al., 2024a; Parachuk, 2022; Barnard, Dom, 2022). Notably, conventional evaluation metrics like token generation throughput or average time-per-output-token provide incomplete insights, as they fail to capture the crucial relationship between token delivery timing and actual user consumption patterns.

2.3 Limitations of Existing Text Streaming Applications

Existing LLM serving primarily relies on two deployment paradigms: on-device and on-server inference. With rapid hardware and software advancements, on-device LLMs have achieved sufficient accuracy levels for many applications (Phan et al., 2025), as evidenced by the integration of Apple Intelligence (Gunter et al., 2024) and Google’s Gemini Nano (Google, 2024) into iOS and Android platforms, where they effectively handle text completion and message composition tasks. While on-device LLMs may still be inadequate for complex tasks (e.g., advanced mathematical reasoning), we focus on the growing category of applications where current on-device models already achieve satisfactory accuracy. For these applications, the challenge is not model capability, but rather the substantial monetary or energy cost demands of LLM inference.

Unfortunately, both serving paradigms face challenges. On-device inference, though enabling faster generation powered by its dedicated resources (Song et al., 2023; Xue et al., 2024b), suffers from extended TTFT for long prompts due to limited processing speeds and substantial energy consumption that scales linearly with response lengths (Li et al., 2024b). For instance, a fully-charged iPhone running a 7B parameter LLM can only operate for less than two hours (Liu et al., 2024d)—insufficient for day-long mobile use.

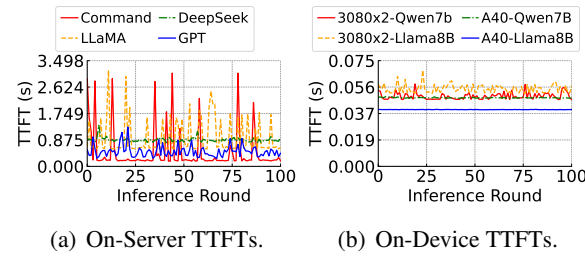


Figure 2: On-device TTFT performance is more stable.

On the other hand, on-server deployments require request batching to amortize costs due to the high resource demands, but this introduces issues like queuing delays, resource contention from batching (Yu et al., 2022; Kwon et al., 2023; Agrawal et al., 2024), and last-hop network latency variations (Li et al., 2024a). Our measurements reveal that these factors can cause significant TTFT spikes for GPT-4-mini, from 0.3 seconds to several seconds during high-load periods.

Given these complementary limitations, we investigate the following research question: *Can a cooperative paradigm be designed to combine on-server and on-device inference to improve QoE while managing both energy and monetary costs?*

3 Characterizing LLM Inference

This section characterizes the LLM inference performance of on-server and on-device paradigms, which informs our design.

We evaluate four commercial streaming LLM APIs: OpenAI’s GPT-4o-mini (OpenAI, 2024), DeepSeek’s DeepSeek-V2.5 (DeepSeek, 2024), Cohere’s Command (Cohere, 2024), and Hyperbolic-hosted LLaMA-3-70b-Instruct (Hyperbolic, 2024). For on-device analysis, we deploy Qwen-2.5-7B-Instruct (Alibaba, 2024) and Llama-3.1-8B-Instruct (Grattafiori et al., 2024) on both server-grade (NVIDIA A40, 48GB) and consumer-grade (dual NVIDIA RTX 3080, denoted as 3080x2) GPUs. We sample 1,000 requests from the Alpaca dataset (Taori et al., 2023), following a Poisson distribution with a mean request arrival interval of 30 seconds.

TTFT characteristics. Our measurements reveal the contrasting TTFT patterns between on-device and on-server inference. As shown in Figure 2, on-device inference exhibits stable TTFTs when processing identical prompts at 60-second intervals, primarily reflecting the prefill duration due

from几百毫秒到超过十秒——远远超过交互式应用理想的几十毫秒延迟 (Mäki-Patola and Härmäläinen, 2004; Žádník et al., 2022)。令牌消耗模式因输出方式而异：在视觉文本场景中，不同人口群体的阅读速度不同，其中大多数 (52%) 年龄在 25-44 岁的人以每秒 4-5 个令牌的速度阅读，而年龄较大的群体一般阅读较慢 (Liu et al., 2024a; Petrov et al., 2024)。音频输出的消耗更为一致，各语言平均为每秒 3-4 个令牌 (Liu et al., 2024a; Petrov et al., 2024)。

2.3 现有文本流应用的局限性

现有的大型语言模型 (LLM) 服务主要依赖两种部署模式：设备端和服务器端推理。随着硬件和软件的快速发展，设备端的 LLM 已经在许多应用中达到了足够的准确性水平 (Phan et al., 2025)，这可以从 Apple Intelligence (Gunter et al., 2024) 和谷歌的 Gemini Nano (Google, 2024) 在 iOS 和 Android 平台上的整合中得到验证，这些模型能够有效处理文本补全和消息撰写任务。尽管设备端 LLM 在处理复杂任务 (如高级数学推理) 方面可能仍不够充分，但我们关注的是当前设备端模型已经能够达到令人满意准确性的逐渐增加的应用类别。对于这些应用，挑战不在于模型能力，而在于 LLM 推理所需的巨额金钱或能源成本。

不幸的是，这两种服务模式都面临挑战。尽管设备端推理能够利用其专用资源实现更快的生成 (Song et al., 2023; Xue et al., 2024b)，但由于处理速度有限且能耗随响应长度线性增加，它在处理长提示时的首次响应时间 (TTFT) 较长 (Li et al., 2024b)。例如，一部充满电的 iPhone 运行 7B 参数的 LLM 时，最多只能运行不到两个小时 (Liu et al., 2024d)——这不足以满足全天移动使用。

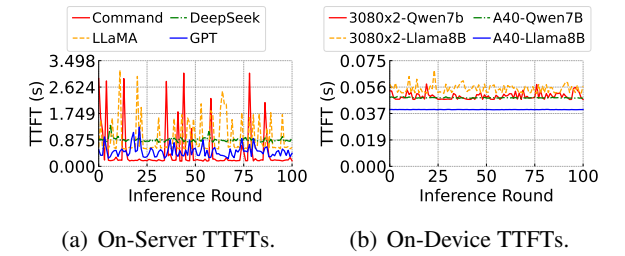


图 2: 设备上的 TTFT 性能更稳定。

另一方面，服务器端部署由于高资源需求，需要进行请求批处理以分摊成本，但这会引入排队延迟、批处理引起的资源竞争 (Yu et al., 2022; Kwon et al., 2023; Agrawal et al., 2024)，以及最后一跳网络延迟变化 (Li et al., 2024a)。我们的测量显示，这些因素可能导致 GPT-4-mini 的首次响应时间 (TTFT) 出现显著波动，在高负载期间从 0.3 秒增加到几秒钟。

鉴于这些互补的限制，我们研究以下问题：是否可以设计一种协作范式，将服务器端和设备端推理结合起来，以在管理能源和金钱成本的同时提升用户体验 (QoE)？

3 大型语言模型推理的特征化

本节描述了服务器端和设备端范式的 LLM 推理性能，这为我们的设计提供了参考。

我们评估了四个商业流媒体大型语言模型 (LLM) API: OpenAI 的 GPT-4o-mini (OpenAI, 2024)、DeepSeek 的 DeepSeek-V2.5 (DeepSeek, 2024)、Cohere 的 Command (Cohere, 2024) 和 Hyperbolic 托管的 LLaMA-3-70b-Instruct (Hyperbolic, 2024)。对于本地设备分析，我们部署了 Qwen-2.5-7B-Instruct (Alibaba, 2024) 和 Llama-3.1-8B-Instruct (Grattafiori et al., 2024)，在服务器级 (NVIDIA A40, 48GB) 和消费级 (双 NVIDIA RTX 3080, 记作 3080x2) GPU 上运行。我们从 Alpaca 数据集 (Taori et al., 2023) 中抽取了 1,000 个请求样本，按照平均请求到达间隔为 30 秒的泊松分布进行采样。

TTFT 特性。我们的测量显示了设备端推理和服务器端推理之间截然不同的 TTFT 模式。如图 2 所示，在以 60 秒间隔处理相同提示时，设备端推理表现出稳定的 TTFT，主要反映了由于预填充导致的时间。

Model	Deployment	Pearson Coef.
Command	Server	0.0142
GPT-4o-mini	Server	0.0236
DeepSeek-V2.5	Server	-0.0273
LLaMA-3-70b-Instruct	Server	0.0402
LLaMA-3.1-8b-Instruct	Device	0.8424

Table 1: Pearson coefficient between prompt length and TTFT in on-server deployment is weak.

to dedicated local hardware resources. In contrast, on-server inference experiences high variations and significant tail latency, attributed to network delays, request queuing, and resource contention.

Unlike previous works that focus solely on prefill latency (e.g., (Gim et al., 2024; Kamahori et al., 2024)) or the sum of queuing and prefill (e.g., (Agrawal et al., 2024; Qin et al., 2024)), we measure user-perceived TTFT as a comprehensive sum of network, queuing, and prefill latencies. Our extensive evaluations across four major LLM providers consistently demonstrate this improvement. This methodology aligns with industry benchmarks, such as Artificial Analysis (Artificial Analysis, 2025)’s continuous monitoring of diverse LLM services. Notably, since most on-device LLMs struggle with prefilling or understanding long prompts, we focus on short prompts, where request queuing delay and batch completion waiting time during generation dominate the overall TTFT.

We summarize the TTFT performance of 1,000 requests in Table 1. We observe that on-device TTFT scales linearly with prompt length due to hardware constraints (Li et al., 2024b), while on-server TTFT shows minimal prompt-length sensitivity through advanced resource scaling (Zhong et al., 2024; Patel et al., 2024; Hu et al., 2024). Note that LLM providers (e.g., Microsoft Azure (Microsoft Azure, 2025), DeepSeek (DeepSeek, 2025), Together.ai (Together AI, 2025), Hyperbolic Labs (Hyperbolic, 2025)) typically do not offer explicit TTFT SLOs, likely due to the high cost and complexity of maintaining such guarantees across diverse models and prompts.

TBT characteristics. TBT characterizes the I/O-bound decode stage latency. Analysis of temporal samples and distributions across six setups (Figure 3) reveals higher TBT variability in on-server inference compared to on-device execution. Moreover, both deployments achieve generation speeds exceeding user consumption rates (§ 2.2), making cooperative serving practical.

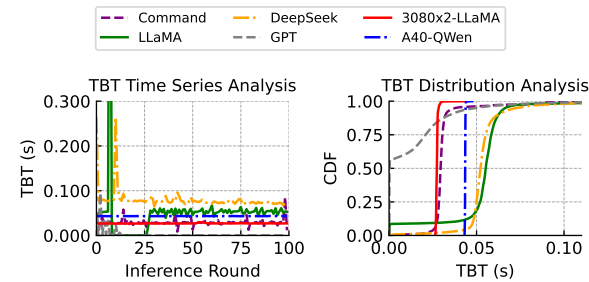


Figure 3: On-device TBT performance is more stable.¹

Opportunities and challenges. Our studies further reveal that as on-device models continue to improve—often fine-tuned for specific tasks (Gunter et al., 2024; Liu et al., 2024d)—they are progressively achieving performance parity with on-server models in popular applications like instruction-following and translation (detailed in §5 and Appendix D). However, deploying these models on-device introduces challenges such as long prefill latency and startup overhead.

On the other hand, our real-world studies of conversational workloads highlight key opportunities: (i) on-server TTFT is largely unpredictable and shows minimal correlation with prompt length, whereas on-device TTFT scales nearly linearly with prompt length and is highly predictable; and (ii) both paradigms achieve token generation speeds that exceed typical user consumption rates.

By integrating these findings—especially the predictable performance of on-device inference and the elastic scaling capabilities of server-based inference—we observe opportunities for optimization in cost-constrained device-server cooperative serving. Dynamic request migration between server and device endpoints during response generation can yield significant cost savings.

4 DiSCo Policies

DiSCo optimizes both QoE and cost through (1) dispatch control that determines where to initiate token generation, and (2) migration control that enables dynamic handoff during generation. The dispatch controller optimizes TTFT by strategically routing requests, while the migration controller maintains consistent TBT while reducing costs.

¹On-server inference, such as in GPT, streams tokens with each packet containing multiple tokens, resulting in near-zero perceived TBTs.

Model	Deployment	Pearson Coef.
Command	Server	0.0142
GPT-4o-mini	Server	0.0236
DeepSeek-V2.5	Server	-0.0273
LLaMA-3-70b-Instruct	Server	0.0402
LLaMA-3.1-8b-Instruct	Device	0.8424

Table 1: Pearson coefficient between prompt length and TTFT in on-server deployment is weak.

专用本地硬件资源。相比之下，服务器推理经历高波动性和显著的尾时延，这归因于网络延迟、请求排队和资源竞争。

与以前仅关注预填充延迟的工作（例如，Gim 等人，；Kamahori 等人，）或排队和预填充总和的工作（例如，Agrawal 等人，；Qin 等人，）不同，我们将用户感知的首次生成时间（TTFT）测量为网络、排队和预填充延迟的综合总和。我们在四个主要大型语言模型提供商上的广泛评估一致显示了这一改进。这种方法与行业基准一致，例如人工分析（Artificial Analysis，）对不同大型语言模型服务的持续监控。值得注意的是，由于大多数本地设备上的大型语言模型在预填充或理解长提示方面存在困难，我们专注于短提示，其中请求排队延迟和生成过程中的批处理完成等待时间主导了整体 TTFT。

我们在表 中总结了 1,000 个请求的 TTFT 性能。我们观察到，由于硬件限制，设备端 TTFT 随提示长度呈线性增长（Li 等人，），而服务器端 TTFT 通过先进的资源扩展对提示长度的敏感度最小（Zhong 等人，；Patel 等人，；Hu 等人，）。注意，LLM 提供者（例如 Microsoft Azure（Microsoft Azure，）、DeepSeek（，）、Together.ai（，）、Hyperbolic Labs（，））通常不提供明确的 TTFT 服务水平目标（SLOs），这可能是因为在不同的条件下维持此类保证的成本高且复杂

TBT 特性。TBT 描述了 I/O 限制的解码阶段延迟。对六种设置下的时间样本和分布进行分析（图 ）显示，服务器端推理的 TBT 变化性高于设备端执行。此外，两种部署方式的生成速度均超过用户消费速率（§ ），使协作服务成为可能。

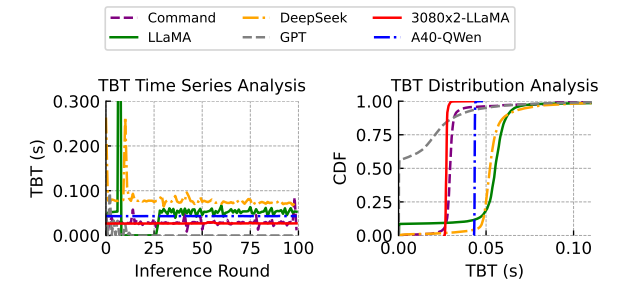


图 3：设备端 TBT 性能更稳定。

机遇与挑战。我们的研究进一步表明，随着设备端模型的不断改进——通常针对特定任务进行微调（Gunter 等。，；Liu 等。，）——它们在流行应用中，如遵循指令和翻译等方面，正逐渐与服务器端模型实现性能对等（详见 § 及附录 ）。然而，在设备端部署这些模型会带来诸如预填充延迟长和启动开销等挑战。

另一方面，我们对真实世界中对话工作负载的研究突出了关键的机会：（i）服务器端的 TTFT 大体上是不可预测的，并且与提示长度的相关性很小，而设备端的 TTFT 几乎与提示长度线性相关，并且高度可预测；（ii）两种范式的令牌生成速度都超过了典型用户的消耗速度。

通过整合这些发现——尤其是在设备端推理的可预测性能和基于服务器的推理的弹性扩展能力——我们观察到了在受成本限制的设备-服务器协同服务中优化的机会。在响应生成过程中在服务器和设备端点之间动态迁移请求可以带来显著的成本节约。

4 政策

通过（1）调度控制来确定在哪里启动令牌生成，以及（2）迁移控制在生成过程中实现动态切换，从而优化了 QoE 和成本。调度控制器通过有策略地路由请求优化了 TTFT，而迁移控制器在降低成本的同时保持了 TBT 的一致性。

服务器端推理，例如在 GPT 中，每个数据包流式传输包含多个令牌的令牌，从而导致几乎为零的感知 TBT。

4.1 Problem Formulation

We propose a unified cost model combining both monetary bills from on-server inference and energy bills from on-device inference. Let c_s^p and c_s^d denote the per-token monetary costs for server prefill and decode phases, respectively, while c_d^p and c_d^d represent the per-token energy costs for device prefill and decode phases. Integration of energy and monetary costs is done by a dynamic exchange rate λ , adjusted by users to reflect their preferences. We offer a user-friendly tunable budget ratio $b \in [0, 1]$, representing the additional cost allowance beyond baseline costs. Our optimization objectives focus on: (1) minimizing both mean and tail TTFT, and (2) maintaining consistent token delivery at a specified pace (i.e., stable TBT).

4.2 Dispatch Controller: Cost-Aware Request Routing

Based on our analysis in §3, server-side TTFT shows weak correlation with prompt length due to various factors (network delay, request queuing, etc.). We model server TTFT as a known distribution, obtained either from server-provided information or device-side profiling. In contrast, device-side TTFT exhibits a linear relationship with prompt length, with the coefficient determined through offline profiling.

Our key insight is that the optimization problem naturally decomposes into two scenarios based on dominant cost factors: device-constrained scenarios where energy consumption is the primary bottleneck, and server-constrained scenarios where API monetary costs dominate. This decomposition enables efficient solutions. The pseudocode for the dispatch controller is attached in Appendix F.

Device-Constrained Optimization. When device costs dominate ($\min(c_d^p, c_d^d) > \max(c_s^p, c_s^d)$), we need to carefully manage device resource usage under a budget constraint $\mathbb{E}[I_d(l)l] \leq b \cdot \mathbb{E}[l]$, where l is the prompt length and $I_d(l)$ indicates device execution. The key challenge is balancing between two goals: leveraging device execution to bound worst-case latency while conserving energy on shorter prompts when possible.

Our solution uses a waiting-time strategy: for each prompt of length l , we first try server execution and wait for time $w(l)$ before potentially starting device execution. This conserves device energy when the server responds quickly. We determine the optimal wait time through a two-phase

approach:

- **Phase 1 (Tail Protection):** We reserve a budget portion α for worst-case scenarios by setting a maximum wait time $w_{tail} = F^{-1}(1 - \min(\alpha, b))$, where $F(\cdot)$ is the server TTFT distribution. This ensures we have device resources ready when server latency exceeds its $(1 - \min(\alpha, b))$ -th percentile.
- **Phase 2 (Average Case):** With the remaining budget $(b - \alpha)$, we set length-dependent wait times:

$$w(l) = \begin{cases} 0 & \text{if } l \leq l_{th} \\ \min(\beta l, w_{tail}) & \text{otherwise} \end{cases} \quad (1)$$

where l_{th} is a threshold below which we start device execution immediately, and β is chosen to satisfy:

$$\int_{l_{th}}^{\infty} (1 - F(\beta l)) \cdot c_d^p \cdot l \cdot p(l) dl = (b - \alpha) \cdot \mathbb{E}[l] \quad (2)$$

This design guarantees worst-case TTFT through w_{tail} while optimizing average performance by adaptively adjusting wait times based on prompt length. Whichever endpoint (server or device) generates the first token continues to the decode phase, while the other terminates.

Server-Constrained Optimization. When server costs dominate ($\max(c_s^p, c_s^d) > \min(c_d^p, c_d^d)$), we need to carefully manage server resource usage under a budget constraint $\mathbb{E}[I_s(l)l] \leq b \cdot \mathbb{E}[l]$, where $I_s(l)$ indicates server execution. Our analysis in §3 shows that device TTFT scales linearly with prompt length as $T_d(l) = kl + c$, while server TTFT has minimal length correlation. This suggests a length-based routing strategy: short prompts run on the device to conserve server budget, while long prompts use both endpoints to minimize TTFT.

We determine the length threshold l_{th} by:

$$\int_0^{l_{th}} l \cdot p(l) dl = (1 - b) \cdot \mathbb{E}[l] \quad (3)$$

This ensures prompts shorter than l_{th} consume exactly $(1 - b)$ fraction of total expected tokens through device-only execution, leaving the remaining longer prompts with sufficient server budget for concurrent execution on both endpoints.

4.1 问题表述

我们提出了一个统一的成本模型，结合了来自服务器端推理的货币账单和来自设备端推理的能源账单。设 c_s^p 和 c_s^d 分别表示服务器预填充和解码阶段的每个 token 货币成本，而 c_d^p 和 c_d^d 分别表示设备端预填充和解码阶段的每个 token 能源成本。能源成本和货币成本的整合通过动态汇率 λ 完成，由用户调整以反映其偏好。我们提供了一个用户友好的可调预算比例 $b \in [0, 1]$ ，表示超出基线成本的额外成本允许。我们的优化目标集中在：(1) 最小化平均和尾部 TTFT，以及 (2) 在指定速度下保持一致的 token 交付（即稳定的 TBT）。

4.2 DispatchController: 成本感知请求路由

根据我们在 § 3 中的分析，服务器端 TTFT 由于各种因素（网络延迟、请求排队等）与提示长度的相关性较弱。我们将服务器 TTFT 建模为已知分布，该分布可以通过服务器提供的信息或设备端分析得到。相比之下，设备端 TTFT 与提示长度呈线性关系，其系数通过离线分析确定。

我们的关键见解是优化问题自然地分解为两种情景，基于主要成本因素：设备受限的情景中，能耗是主要瓶颈；服务器受限的情景中，API 的金钱成本占主导。此分解使高效解决方案成为可能。调度控制器的伪代码附在附录 F。

设备受限优化。当设备成本占主导（ $\min(c_d^p, c_d^d) > \max(c_s^p, c_s^d)$ ）时，我们需要在预算限制下仔细管理设备资源使用 $\mathbb{E}[I_d(l)l] \leq b \cdot \mathbb{E}[l]$ ，其中 $I_d(l)$ 表示设备执行。我们在 § 3 中的分析显示，设备 TTFT 随提示长度线性增长，即 $T_d(l) = kl + c$ ，而服务器 TTFT 与长度几乎没有相关性。这表明可以采用基于长度的路由策略：短提示在设备上运行以节省服务器预算，而长提示同时使用两个端点以最小化 TTFT。

我们的解决方案使用等待时间策略：对于每个长度为 l 的提示，我们首先尝试服务器执行，并等待时间 $w(l)$ ，然后可能才开始设备执行。当服务器响应迅速时，这可以节省设备的能源。我们通过两阶段确定最佳等待时间

方法：

阶段 1（尾部保护）：我们为最坏情况保留了一部分预算 α ，通过设置最大等待时间 $w_{tail} = F^{-1}(1 - \min(\alpha, b))$ ，其中 $F(\cdot)$ 是服务器 TTFT 分布。这确保当服务器延迟超过其 $(1 - \min(\alpha, b))$ 百分位时，我们有设备资源准备就绪。

第二阶段（平均情况）：在剩余预算 $(b - \alpha)$ 下，我们设置了依赖长度的等待时间：

$$w(l) = \begin{cases} 0 & \text{if } l \leq l_{th} \\ \min(\beta l, w_{tail}) & \text{otherwise} \end{cases} \quad (1)$$

其中 l_{th} 是一个阈值，低于该阈值我们会立即开始设备执行，并且 β 的选择应满足：

$$\int_{l_{th}}^{\infty} (1 - F(\beta l)) \cdot c_d^p \cdot l \cdot p(l) dl = (b - \alpha) \cdot \mathbb{E}[l] \quad (2)$$

该设计通过 w_{tail} 保证最坏情况下 TTFT，同时通过根据提示长度自适应调整等待时间来优化平均性能。无论哪个端点（服务器或设备）生成第一个令牌，都将继续进入解码阶段，而另一个端点将终止。

服务器受限优化。当服务器成本占主导（ $\max(c_s^p, c_s^d) > \min(c_d^p, c_d^d)$ ）时，我们需要在预算约束下仔细管理服务器资源使用 $\mathbb{E}[I_s(l)l] \leq b \cdot \mathbb{E}[l]$ ，其中 $I_s(l)$ 表示服务器执行。我们在 § 3 中的分析显示，设备 TTFT 随提示长度线性增长，即 $T_d(l) = kl + c$ ，而服务器 TTFT 与长度几乎没有相关性。这表明可以采用基于长度的路由策略：短提示在设备上运行以节省服务器预算，而长提示同时使用两个端点以最小化 TTFT。

我们通过以下方式确定长度阈值 l_{th} ：

$$\int_0^{l_{th}} l \cdot p(l) dl = (1 - b) \cdot \mathbb{E}[l] \quad (3)$$

这确保了长度小于 l_{th} 的提示通过仅设备执行消耗总预期令牌的准确 $(1 - b)$ 分数，从而为较长的剩余提示保留充足的服务器预算，以便在两个端点上并发执行。

4.3 Migration Controller: Cost-Efficient Token Delivery

When both endpoints process a request, the constrained endpoint may win the prefill phase but incur higher decode costs. In such cases, we can migrate token generation to the other endpoint to reduce total cost while maintaining quality.

Theoretical Migration Framework. The token-level migration protocol ensures seamless handoffs by leveraging the gap between token generation (r_g) and consumption (r_c) rates. The migration trigger is determined by comparing the cost savings against migration overhead:

$$C_{\text{migration}} = \Delta c_{\text{decode}} \cdot l_{\text{remaining}} > \text{Overhead}_{\text{migration}} \quad (4)$$

where Δc_{decode} is the per-token cost difference between endpoints and $l_{\text{remaining}}$ represents the expected remaining sequence length. This formulation ensures that migration only occurs when the projected cost savings exceed the overhead of transferring control between endpoints.

Efficient Token Transfer. When endpoints share the same vocabulary, we transmit token IDs rather than complete token representations. Empirical analysis using the Alpaca dataset (Taori et al., 2023) with the cl100k_base tokenizer (OpenAI, 2022) (used by GPT-3.5/4) demonstrates that token ID transmission versus UTF-8 encoded text yields 35.62% reduction in data volume when using minimum byte encoding (3 bytes per token), and 54.40% reduction in data volume when using minimum bit encoding (17 bits per token). These efficiency gains are particularly valuable in bandwidth-constrained environments, helping to minimize latency during migration.

Additionally, we avoid transferring intermediate states (e.g., attention key-value cache) for two practical reasons: (1) endpoints often employ different model architectures optimized for their respective hardware, making state transfer incompatible, and (2) intermediate state transfer would incur significant network overhead. For models with different vocabularies, we first convert tokens to text before re-tokenizing on the target model to ensure semantic consistency.

Migration is triggered when the projected cost savings exceed overhead:

$$C_{\text{migration}} = \Delta c_{\text{decode}}^d \times l_{\text{remaining}} \quad (5)$$

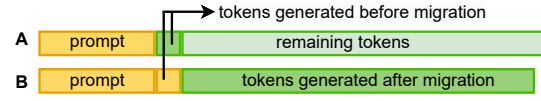


Figure 4: Token generation migration between endpoints. Row A shows the original sequence on the source endpoint, while Row B shows the sequence after migration to the target endpoint, maintaining consistent token delivery while reducing cost.

where $\Delta c_{\text{decode}}^d = |c_s^d - c_d^d|$ and $l_{\text{remaining}}$ denote the per-token decode cost difference between endpoints, and the expected remaining sequence length, respectively.

Buffer-Based Migration Protocol. To ensure smooth token delivery during migration, we introduce a token buffer that leverages the natural gap between token generation speed (r_g tokens/s) and human consumption rate (r_c tokens/s, typically $r_g > r_c$). The buffer size is set to:

$$B = r_c \times t_m \quad (6)$$

where t_m is the estimated migration overhead time. Migration begins only when the buffer contains enough tokens (B) to overshadow the migration latency. Importantly, the source endpoint stops generating new tokens once the buffer is filled, preventing any potential conflicts or branching during the transition. This ensures deterministic token delivery as the target endpoint takes over generation.

As shown in Figure 4, this design enables seamless handoff: the source endpoint (Row A) continues generation until the buffer is filled, then stops to allow the target endpoint (Row B) to take over, ensuring uninterrupted token delivery to users despite the underlying endpoint transition.

5 Evaluation

Through extensive experimentation with four production-grade LLM services and state-of-the-art open-source models, we demonstrate *DiSCo*'s exceptional performance. Our rigorous evaluation spanning diverse deployment scenarios reveals that *DiSCo* delivers remarkable improvements, reducing both mean TTFT (6-78%) and tail TTFT (11-52%) while achieving cost savings of up to 83.6%.

5.1 Evaluation Setup

Testbeds and Workloads. Our testbed is a server with 4 NVIDIA A40 GPUs, each with 48GB of memory. We evaluate *DiSCo* using both commercial LLM traces and on-device deployments. For

4.3 迁移控制器：高效的代币传输

当两个端点都处理请求时，受约束的端点可能在预填充阶段获胜，但会产生更高的解码成本。在这种情况下，我们可以将令牌生成迁移到另一个端点，以在保持质量的同时降低总成本。

理论迁移框架。令牌级迁移协议通过利用令牌生成率（ ）与消耗率（ ）之间的差距，确保无缝切换。迁移触发器是通过比较节约成本与迁移开销来确定的：

$$C_{\text{migration}} = \Delta c_{\text{decode}} \cdot l_{\text{remaining}} > \text{Overhead}_{\text{migration}} \quad (4)$$

其中 是端点之间每个令牌的成本差异， 表示预期的剩余序列长度。该公式确保只有当预计的成本节约超过在端点之间转移控制的开销时，迁移才会发生。

高效的令牌传输。当端点共享相同的词汇表时，我们传输的是令牌 ID 而不是完整的令牌表示。使用 Alpaca 数据集（Taori 等人）和 cl100k_base 分词器（ ）（GPT-3.5/4 使用）进行的实证分析表明，令牌 ID 传输相比 UTF-8 编码文本，在使用最小字节编码（每个令牌 3 字节）时，数据量减少 35.62%，在使用最小位编码（每个令牌 17 位）时，数据量减少 54.40%。这些效率提升在带宽受限的环境中尤为有价值，有助于在迁移过程中最小化延迟。

此外，我们避免传输中间状态（例如，注意力键值缓存）有两个实际原因：（1）端点通常采用针对各自硬件优化的不同模型架构，使状态传输不兼容；（2）中间状态的传输会带来显著的网络开销。对于具有不同词汇表的模型，我们首先将标记转换为文本，然后在目标模型上重新标记，以确保语义一致性。

当预计节省的成本超过开销时，就会触发迁移：

$$C_{\text{migration}} = \Delta c_{\text{decode}}^d \times l_{\text{remaining}} \quad (5)$$

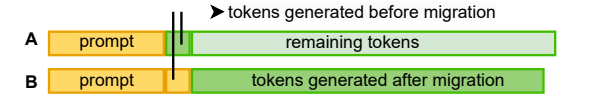


图 4：终端之间的令牌生成迁移。A 行显示源终端上的原始序列，而 B 行显示迁移到目标终端后的序列，在保持令牌传递一致的同时降低了成本。

其中 和 de——分别注意各端点之间每个标记解码成本的差异，以及预期剩余序列长度。

基于缓冲的迁移协议。为了确保迁移过程中令牌的顺利传递，我们引入了一个令牌缓冲区，该缓冲区利用令牌生成速度（ 令牌 / 秒）与人类消耗速度（ 令牌 / 秒，通常 ）之间的自然间隙。缓冲区大小设置为：

$$B = r_c \times t_m \quad (6)$$

其中 是估计的迁移开销时间。迁移仅在缓冲区包含足够的标记（ ）以覆盖迁移延迟时才开始。重要的是，一旦缓冲区填满，源端点会停止生成新的标记，从而防止在过渡期间发生任何潜在的冲突或分支。这确保了在目标端点接管生成时标记的确定性传递。

如图所示，这一设计实现了无缝切换：源端点（行 A）继续生成直到缓冲区填满，然后停止，以允许目标端点（行 B）接管，从而确保尽管底层端点发生切换，用户仍能连续获取令牌。

5 评估

通过对四个生产级大型语言模型服务和最先进的开源模型进行广泛实验，我们展示了 的卓越性能。我们涵盖多种部署场景的严格评估显示， 带来了显著的改进，平均 TTFT 降低了 6-78%，尾部 TTFT 降低了 11-52%，同时实现了高达 83.6% 的成本节省。

5.1 评估设置

测试平台和工作负载。我们的测试平台是一台配备 4 个 NVIDIA A40 GPU 的服务器，每个 GPU 有 48GB 的内存。我们使用商业 LLM 跟踪数据和设备端部署来评估 。

server-side evaluation, we collect traces from four production services: OpenAI’s GPT-4o-mini (OpenAI, 2024), DeepSeek-V2.5 (DeepSeek, 2024), Cohere’s Command (Cohere, 2024), and Hyperbolic-hosted LLaMA-3-70b-Instruct (Hyperbolic, 2024). For on-device evaluation, we test three representative device-model configurations (Li et al., 2024b): Pixel 7 Pro with Bloom 1.1B (31.32/13.93 tokens/s for prefill/decode), Pixel 7 Pro with Bloom 560M (51.80/20.14 tokens/s), and Xiaomi 14 with Qwen 1.5 0.5B (79.90/21.47 tokens/s). These configurations span different compute-capability trade-offs in mobile environments. For end-to-end cost comparison, we quantify server costs using commercial API token pricing and device costs using FLOPs-based energy consumption. The detailed cost analysis can be found in Appendix E.

Baselines. We compare *DiSCo* with four on-server, on-device, and cooperative deployments:

- *vLLM* (Kwon et al., 2023): Processes all requests using remote server-based deployment.
- *llama.cpp* (Gerganov, 2024): Processes all requests using local device-based deployment.
- *Stoch-S*: A server-constrained approach that randomly routes requests to the device while capping the server budget.
- *Stoch-D*: A device-constrained approach that randomly routes requests to the server while capping the device budget.

For end-to-end cost comparison, we include two additional baselines: *DiSCo-D w/o Migration* and *DiSCo-S w/o Migration*.

Metrics. We evaluate the system performance using both TTFT and TBT, including their mean and tail values. They are analyzed across varying cost budgets, defined as the ratio of input tokens processed by the constrained endpoint (device or server) to the total input tokens. For each experiment, we report the mean value over 10 runs.

5.2 End-to-end Performance

***DiSCo* improves TTFT performance.** Figure 6 and Table 2 show that *DiSCo* significantly outperforms baseline methods in both device- and server-constrained settings, showing improvements across mean and tail (P99) TTFT metrics for various services, including GPT, LLaMA, DeepSeek, and Command. In the GPT experiments, *DiSCo* demonstrates particularly notable tail latency reductions, decreasing P99 TTFT by up to 40% relative

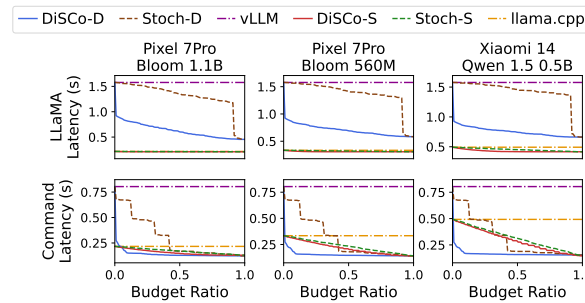


Figure 5: Mean TTFT reduction of *DiSCo* remains significant on DiffusionDB trace.

Tail TTFT Reduction					
Platform	Constraint	Pixel 7Pro B-1.1B	Pixel 7Pro B-560M	Xiaomi 14 Q-0.5B	
GPT	Server	23.85%	37.41%	44.04%	
	Device	26.39%	21.48%	16.32%	
LLaMA	Server	11.08%	23.09%	26.29%	
	Device	35.67%	29.30%	21.29%	
DeepSeek	Server	0.00%*	3.88%	15.53%	
	Device	30.91%	28.01%	25.08%	
Command	Server	47.93%	50.93%	52.23%	
	Device	34.78%	31.53%	24.42%	

Table 2: Average reduction of tail TTFT compared to stochastic dispatching across the whole cost budget range. Devices include Pixel 7 Pro and Xiaomi 14, while models include Bloom-1.1B, Bloom-560M, and Qwen-1.5-0.5B. (*Tail TTFT remains constant.)

to Stochastic dispatching across all device configurations, while mean TTFT is also reduced substantially, with reductions between 20-30% across diverse budget ratios. In the LLaMA setup, we observe a unique trade-off pattern. For budget ratios below 20% when the device is the constrained endpoint, *DiSCo* exhibits a slightly higher mean TTFT than the baseline. This outcome is intentional, as *DiSCo* prioritizes tail latency reduction in low-budget scenarios, yielding substantial gains in P99 TTFT—reducing tail latency by up to 50%.

DeepSeek and Command experiments demonstrate similar patterns of improvement as the previous two traces, with *DiSCo* consistently outperforming baseline approaches. In the DeepSeek scenario, *DiSCo* maintains stable latency even as the budget ratio increases, whereas the baseline systems show increasing latency variance.

***DiSCo* retains TBT performance while lowering the cost.** Table 3 evaluates *DiSCo*’s TBT performance across various traces under both server and device constraints. For requests involving migration, we measure two key metrics: the average number of migrations per request and the tail (P99) TBT

服务器端评估, 我们从四个生产服务中收集跟踪数据: OpenAI 的 GPT-4o-mini (OpenAI)、DeepSeek-V2.5 (DeepSeek)、Cohere 的 Command (Cohere) 和 Hyperbolic 托管的 LLaMA-3-70b-Instruct (Hyperbolic)。对于设备端评估, 我们测试了三种具有代表性的设备 - 模型配置 (Li 等): Pixel 7 Pro 配备 Bloom 1.1B (预填充 / 解码速率为 31.32/13.93 tokens/s)、Pixel 7 Pro 配备 Bloom 560M (速率为 51.80/20.14 tokens/s)、以及小米 14 配备 Qwen 1.5 0.5B (速率为 79.90/21.47 tokens/s)。这些配置覆盖了移动环境中不同的计算能力折衷。为了端到端的成本比较, 我们使用商业 API 的 token 定价来量化服务器成本, 并使用基于 FLOPs 的能耗来评估设备成本。详细的成本分析

基线。 我们将 *DiSCo* 与四种服务器端、设备端和协作部署进行比较:

- *vLLM* (Kwon 等): 使用基于远程服务器的部署处理所有请求。
- *llama.cpp* (Gerganov): 使用本地设备部署处理所有请求。
- *Stoch-S*: 一种受服务器限制的方法, 它在限制服务器预算的同时随机将请求路由到设备。
- *Stoch-D*: 一种受设备限制的方法, 它在随机将请求路由到服务器的同时限制设备预算。

为了端到端的成本比较, 我们包含了两个额外的基线: *DiSCo-D* 不带迁移和 *DiSCo-S* 不带迁移。

指标。 我们使用 TTFT 和 TBT 评估系统性能, 包括它们的平均值和尾部值。它们在不同成本预算下进行分析, 成本预算定义为受限端点 (设备或服务器) 处理的输入标记与总输入标记的比率。对于每个实验, 我们报告 10 次运行的平均值。

5.2 端到端性能

提升了 TTFT 性能。 图 6 和表 2 显示在设备和服务器受限环境中显著优于基线方法, 在各种服务 (包括 GPT、LLaMA、DeepSeek 和 Command) 的平均和尾部 (P99) TTFT 指标上均有提升。在 GPT 实验中, *DiSCo* 在尾部延迟方面表现尤为突出, 相比基线 P99 TTFT 最多降低了 40%。

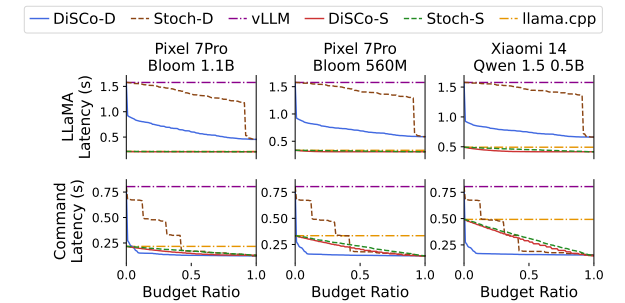


图 5: 的平均 TTF 减少在 DiffusionDB 跟踪上仍然显著。

Tail TTFT Reduction					
Platform	Constraint	Pixel 7Pro B-1.1B	Pixel 7Pro B-560M	Xiaomi 14 Q-0.5B	
GPT	Server	23.85%	37.41%	44.04%	
	Device	26.39%	21.48%	16.32%	
LLaMA	Server	11.08%	23.09%	26.29%	
	Device	35.67%	29.30%	21.29%	
DeepSeek	Server	0.00%*	3.88%	15.53%	
	Device	30.91%	28.01%	25.08%	
Command	Server	47.93%	50.93%	52.23%	
	Device	34.78%	31.53%	24.42%	

表 2: 与随机调度相比, 在整个成本预算范围内尾部 TTFT 的平均减少量。设备包括 Pixel 7 Pro 和小米 14, 而模型包括 Bloom-1.1B、Bloom-560M 和 Qwen-1.5-0.5B。(*尾部 TTFT 保持不变。)

对所有设备配置进行随机调度, 同时平均 TTFT 也显著降低, 在不同的预算比例下减少幅度在 20%-30% 之间。在 LLaMA 设置中, 我们观察到一种独特的权衡模式。当预算比例低于 20% 且设备是受限端点时, *DiSCo* 的平均 TTFT 略高于基准值。这个结果是有意为之的, 因为在低预算场景下优先降低尾部延迟, 从而在 P99TTFT 上取得显著收益——尾部延迟最多可减少 50%。

DeepSeek 和 Command 实验显示了与之前两个实验类似的改进模式, *DiSCo* 始终优于基线方法。在 DeepSeek 场景中, 即使在预算比例增加时仍保持稳定的延迟, 而基线系统的延迟方差则有所增加。

在降低成本的同时保持了 TBT 性能。 表 3 评估了 *DiSCo* 在各种跟踪下, 服务器和设备约束条件下的 TBT 性能。对于涉及迁移的请求, 我们衡量两个关键指标: 每个请求的平均迁移次数以及尾部 (P99) TBT

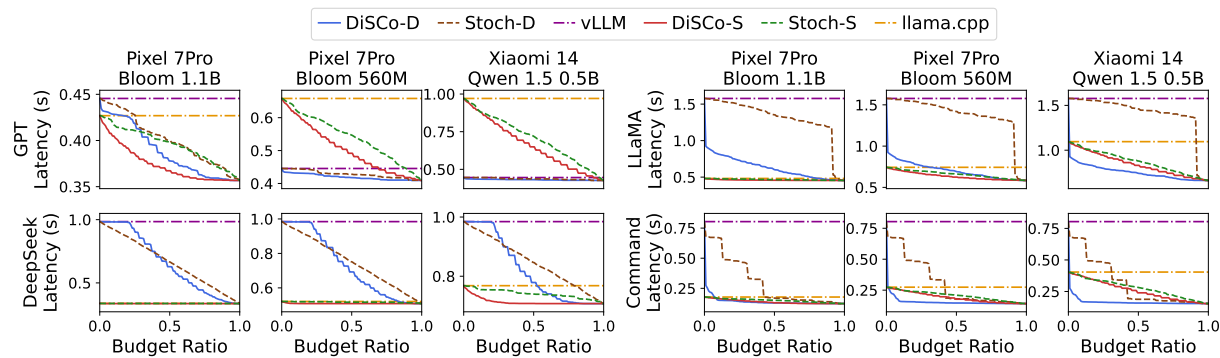


Figure 6: Mean TTFT tested using four traces. *DiSCo* achieves superior TTFT performance than the baselines.

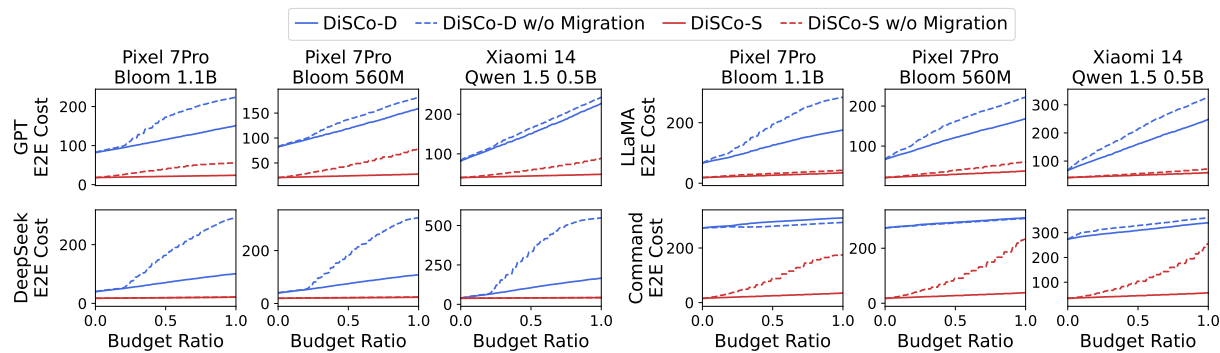


Figure 7: The migration mechanism in *DiSCo* achieves superior end-to-end cost.

Trace	Constraint	Mean delay_num	P99 delay_num	TBT P99
GPT	Server	4.21	9.40	0.209
	Device	6.59	6.59	0.217
LLaMA	Server	5.53	11.00	0.209
	Device	10.01	10.01	0.217
DeepSeek	Server	8.13	11.00	0.209
	Device	17.17	17.17	0.217
Command	Server	3.25	8.00	0.209
	Device	8.54	8.54	0.217

Table 3: Performance metrics for different models under server and device constraints, showing the number of delayed tokens during migration and TBT (Time Between Tokens) P99 statistics. The average is computed over the requests that have performed the migration.

latency. Results show that while migrations delay only a negligible number of tokens compared to typical generation lengths of hundreds or thousands of tokens, they do not impact the perceived token delivery smoothness, demonstrating *DiSCo*'s ability to maintain consistent streaming performance even during endpoint transitions.

As shown in Figure 7, our token-level migration mechanism substantially reduces the end-to-end cost across all evaluated scenarios. For device-constrained cases (*DiSCo-D*), the migration mechanism achieves up to 72.7% cost reduction compared to the non-migration baseline, with the improvement being most significant at higher budget

ratios. Similarly, in server-constrained scenarios (*DiSCo-S*), the cost reduction reaches 83.6%, particularly evident in DeepSeek and Command model deployments. These significant cost reductions are consistently observed across device-model pairs.

5.3 Performance Breakdown and Ablation Study

Impact of Prompt Arrival Interval. To evaluate our system under realistic workload patterns, we conduct experiments using stratified sampling based on request arrival rate from DiffusionDB (Wang et al., 2022). Specifically, we select traces from ten users across different activity levels to capture diverse interaction patterns. We pair these real-world request intervals with prompts randomly drawn from the Alpaca dataset (Taori et al., 2023). The results shown in Figure 5 demonstrate that *DiSCo*'s performance advantages persist across varied user activity patterns.

Quality of Generated Responses. We conduct comprehensive experiments using instruction-following tasks on multiple model configurations. We employ three LLM-based judges (GPT-4o, Gemini1.5-pro, and QWen2.5-72b) to assess the response quality, and examine two representative migration scenarios: from a smaller to larger model (3B-7B) and vice versa (7B-3B). Figure 8 shows

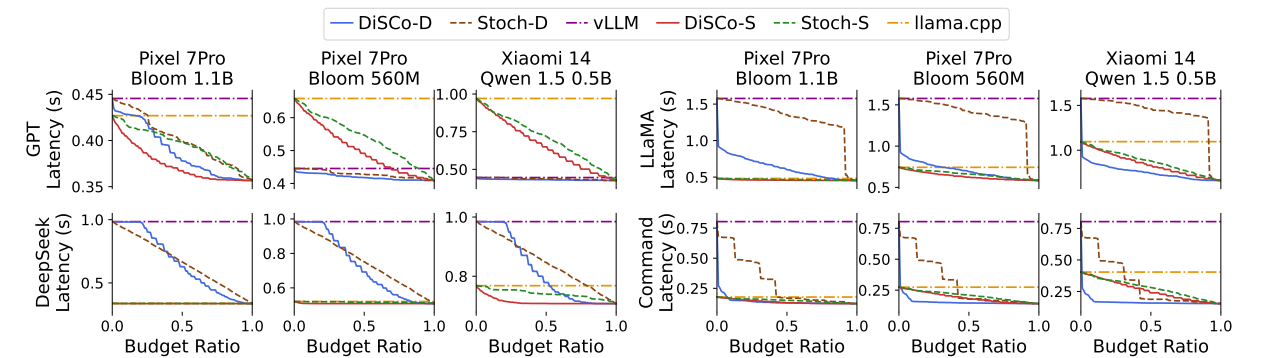


Figure 6: 平均 TTFT 使用四个追踪进行测试。的 TTFT 性能优于基线方法。

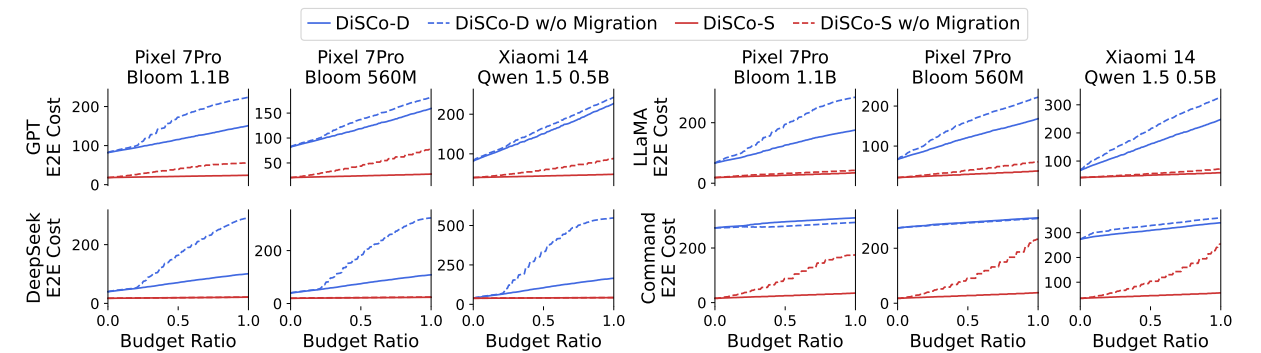


Figure 7: The *DiSCo* 中的迁移机制实现了卓越 end-to-end cost.

Trace	Constraint	Mean delay_num	P99 delay_num	TBT P99
GPT	Server	4.21	9.40	0.209
	Device	6.59	6.59	0.217
LLaMA	Server	5.53	11.00	0.209
	Device	10.01	10.01	0.217
DeepSeek	Server	8.13	11.00	0.209
	Device	17.17	17.17	0.217
Command	Server	3.25	8.00	0.209
	Device	8.54	8.54	0.217

表 3: 在服务器和设备约束下不同模型的性能指标, 显示迁移过程中延迟的令牌数量和 TBT (令牌间时间) P99 统计。平均值是对已执行迁移的请求计算的。

延迟。结果表明, 与通常数百或数千个令牌的生成长度相比, 迁移仅延迟了极少数令牌, 但它们不会影响感知到的令牌传输平滑性, 这展示了在端点过渡期间保持一致流式性能的能力。

如图 所示, 我们的令牌级迁移机制在所有评估的场景中显著降低了端到端成本。对于受设备限制的情况 (*-D*), 与非迁移基线相比, 该迁移机制最多可实现 72.7% 的成本降低, 且在较高预算下改进最为显著

比率。类似地, 在服务器受限的情况下 (*-S*), 成本降低达到了 83.6%, 在 DeepSeek 和 Command 模型部署中尤为明显。这些显著的成本降低在各设备 - 模型组合中均有一致的观察到。

5.3 性能分解与消融研究

提示到达间隔的影响。为了在真实工作负载模式下评估我们的系统, 我们使用基于 DiffusionDB 请求到达率的分层抽样进行实验 (Wang 等人,)。具体来说, 我们从不同活跃度的十位用户中选择跟踪, 以捕捉多样的交互模式。我们将这些现实世界的请求间隔与随机抽取自 Alpaca 数据集的提示配对 (Taori 等人,)。如图 所示, 结果表明 在各种用户活动模式下的性能优势仍然存在。

生成回应的质量。我们在多种模型配置上使用遵循指令的任务进行全面实验。我们采用三种基于大型语言模型的评审 (GPT-4o、Gemini1.5-pro 和 QWen2.5-72b) 来评估回应质量, 并检查两种具有代表性的迁移场景: 从较小模型到大模型 (3B-7B) 及反向 (7B-3B)。图 显示

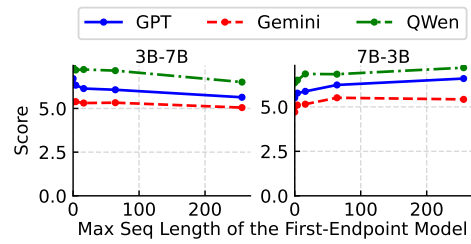


Figure 8: Response quality evaluation. Each subplot represents a distinct model pair configuration (e.g., 3B-7B indicates migration from a 3B to a 7B model). The x-axis shows the maximum sequence length processed by the first endpoint before migration, while the y-axis shows the quality scores assigned by different LLM judges. Results demonstrate consistent quality preservation across various migration scenarios.

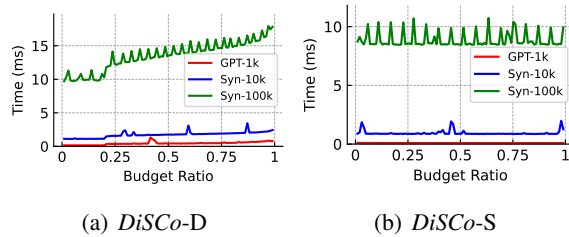


Figure 9: *DiSCo*'s overhead is trivial and can scale well.

that *DiSCo* maintains quality scores across different sequence lengths, migration patterns, and judges. Detailed results are presented in Appendix D.

Scalability Analysis. We conducted comprehensive performance evaluations of *DiSCo*-D and *DiSCo*-S on a MacBook Pro with M1 processor, using both synthetic datasets and a real-world GPT trace of 1,000 records, across target frequencies from 0 to 1. To generate synthetic data that accurately reflects real-world scenarios, we fitted log-normal distributions to the prompt lengths and TTFT from the real trace by following the mean and standard deviation of the logarithm. As shown in Figure 9, for *DiSCo*-S, the execution time showed remarkable efficiency: 0.128 ms for the real trace with 1K samples, scaling to just 0.969 ms and 9.082 ms for synthetic datasets of 10K and 100K samples, respectively. *DiSCo*-D, while being more computationally intensive, still maintained practical performance levels: 0.486 ms, 1.741 ms, and 14.856 ms for 1K, 10K, and 100K samples, respectively.

6 Conclusions

This paper introduces *DiSCo*, a device-server cooperative scheduler that addresses QoE and cost challenges in LLM serving for real-time conversational

applications for end users. *DiSCo* uses cost-aware scheduling and token-level migration to dynamically optimize TTFT and TBT across device and server endpoints. Our evaluations on real-world traces from platforms like GPT and DeepSeek show that *DiSCo* significantly improves both TTFT and TBT while reducing costs.

7 Limitations

While *DiSCo* demonstrates significant improvements in LLM serving efficiency, we acknowledge several important limitations of our current work:

Model Accuracy. We focus on scenarios where on-device LLMs achieve sufficient accuracy for target applications. While this covers many common use cases, *DiSCo* may not be suitable for complex reasoning tasks (Guo et al., 2025).

Privacy Protection. While privacy protection is a key consideration in model selection (Chen et al., 2024; Liu et al., 2024b), *DiSCo* currently assumes users are comfortable with both on-device and on-server deployments.

Energy Modeling. For device energy consumption, we use a linear energy model based on FLOPs. Real-world device energy consumption patterns can be more complicated, varying on factors such as battery state, temperature, and concurrent workloads (Hoque et al., 2015).

Scalability. Our current implementation and evaluation focus on single-device scenarios. Extending *DiSCo* to handle multi-device collaborative serving presents additional challenges in terms of coordination overhead and resource allocation that warrant further investigation (Niu et al., 2025).

8 Ethical Considerations

Our work focuses solely on optimizing the efficiency of LLM serving systems through device-server collaboration and does not introduce new language generation capabilities or content. All experiments were conducted using publicly available models and datasets. While our work may indirectly benefit the accessibility of LLM services by reducing costs and improving performance, we acknowledge that broader ethical considerations around LLM deployment and usage are important but outside the scope of this technical contribution.

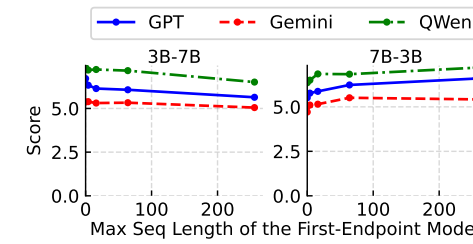


图 8: 响应质量评估。每个子图表示一个不同的模型对配置 (例如, 3B-7B 表示从 3B 模型迁移到 7B 模型)。x 轴显示在迁移前由第一个端点处理的最大序列长度, 而 y 轴显示不同 LLM 评审给出的质量评分。结果显示, 在各种迁移场景中质量保持一致。

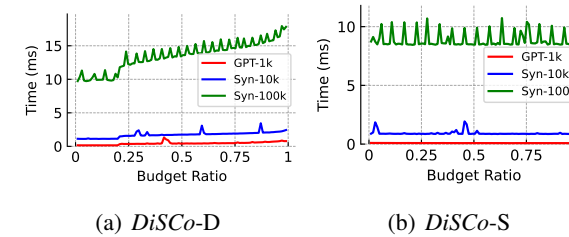


图 9: 开销微不足道, 并且可以很好地扩展。

那 在不同序列长度、迁移模式和评审之间保持质量评分。一些详细结果展示在附录 中。

可扩展性分析。我们对 -D 和 -S 在 MacBook Pro (M1 处理器) 上进行了全面的性能评估, 使用了合成数据集和包含 1,000 条记录的真实 GPT 跟踪数据, 目标频率范围从 0 到 1。为了生成准确反映真实场景的合成数据, 我们通过对真实跟踪数据的提示长度和 TTFT 进行对数正态分布拟合, 依据对数的均值和标准差进行处理。如图 所示, 对于 -S, 其执行时间显示出显著的高效性: 真实跟踪 1K 样本耗时 0.128 毫秒, 合成数据集 10K 和 100K 样本分别仅需 0.969 毫秒和 9.082 毫秒。-D 虽然计算负担更重, 但仍保持了实际可用的性能水平: 1K、10K 和 100K 样本的执行时间分别为 0.486 毫秒、1.741 毫秒和 14.856 毫秒。

6 结论

本文介绍了 , 一种设备 - 服务器协作调度器, 解决了实时对话中大语言模型 (LLM) 服务的用户体验 (QoE) 和成本挑战

面向终端用户的应用程序。使用成本感知调度和令牌级迁移, 在设备和服务器端动态优化 TTFT 和 TBT。我们在来自类似 GPT 和 DeepSeek 的平台真实迹上的评估表明, 在降低成本的同时显著提升了 TTFT 和 TBT。

7 个限制

虽然 在大型语言模型 (LLM) 服务效率方面展示了显著的改进, 但我们承认当前工作的几个重要局限性:

模型准确性。我们专注于设备端 LLM 能够达到目标应用足够准确性的场景。虽然这涵盖了许多常见的用例, 可能不适用于复杂的推理任务 (Guo 等)。

隐私保护。虽然隐私保护是模型选择中的一个关键考虑因素 (陈等。 , ; 刘等,), 目前假设用户对设备端和服务器端部署都感到舒适。

能源建模。对于设备能耗, 我们使用基于 FLOPs 的线性能量模型。现实设备的能耗模式可能更为复杂, 会因电池状态、温度和并发工作负载等因素而有所不同 (Hoque 等人)。

可扩展性。我们当前的实现和评估集中在单设备场景。将 *DiSCo* 扩展到处理多设备协作服务在协调开销和资源分配方面提出了额外的挑战, 这需要进一步的研究 (Niu 等人,)。

8 道德考量

我们的工作完全专注于通过设备 - 服务器协作优化大型语言模型 (LLM) 服务系统的效率, 并不引入新的语言生成能力或内容。所有实验均使用公开可用的模型和数据集进行。虽然我们的工作可能通过降低成本和提升性能间接有利于 LLM 服务的可及性, 但我们承认, 关于 LLM 部署和使用的更广泛伦理考量是重要的, 但超出了本技术贡献的范围。

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Tam-ing throughput-latency tradeoff in llm inference with sarathi-serve. In *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation, 2024, Santa Clara*.

Alibaba. 2024. [Qwen2.5 family](#). Accessed on 21 Sep 2024.

Artificial Analysis. 2025. Artificial analysis: Ai model performance benchmarking. <https://artificialanalysis.ai>. Accessed: 2025-05-17.

Barnard, Dom. 2022. [Average speaking rate and words per minute](#). Accessed on 21 Sep 2024.

Marc Brysbaert. 2019. How many words do we read per minute? a review and meta-analysis of reading rate. *Journal of memory and language*.

Daihang Chen, Yonghui Liu, Mingyi Zhou, Yanjie Zhao, Haoyu Wang, Shuai Wang, Xiao Chen, Tegawend  F Bissyand , Jacques Klein, and Li Li. 2024. Llm for mobile: An initial roadmap. *ACM Transactions on Software Engineering and Methodology*.

Lingjiao Chen, Matei Zaharia, and James Zou. 2023. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.

Cohere. 2024. [Command model](#). Accessed on 21 Sep 2024.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher R . 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359.

Databricks. 2023. LLM Inference Performance Engineering: Best Practices. <https://www.databricks.com/blog/\llm-inference-performance-engineering-best-practices>. [Online; accessed 27-September-2024].

DeepSeek. 2024. [Deepseek-v2.5 model](#). Accessed on 21 Sep 2024.

DeepSeek. 2025. Deepseek: Open-source large language models. <https://www.deepseek.com>. Accessed: 2025-05-17.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332.

Ali Diba, Vivek Sharma, Ali Pazandeh, Hamed Pirsiavash, and Luc Van Gool. 2017. Weakly supervised cascaded convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 914–922.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. [Hybrid llm: Cost-efficient and quality-aware query routing](#).

Angela Fan, Mike Lewis, and Yann Dauphin. 2018. Hierarchical neural story generation. *arXiv preprint arXiv:1805.04833*.

Georgi Gerganov. 2024. [llama.cpp](#). Accessed on 21 Sep 2024.

In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338.

Google. 2024. [Gemini nano](#). Accessed on 21 Sep 2024.

Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc’Aurelio Ranzato, Francisco Guzm n, and Angela Fan. 2022. The flores-101 evaluation benchmark for low-resource and multilingual machine translation. *Transactions of the Association for Computational Linguistics*, 10:522–538.

Grand View Research. 2023. [Large language model \(llm\) market size, share & trends analysis report by component, by application, by enterprise size, by end-use, by region, and segment forecasts, 2023 - 2030](#). Grand View Research.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Al-lonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzm n, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Is-han Misra, Ivan Evtimov, Jack Zhang, Jade Copet,

参考文献 Amey Agrawal, Nitin Kedia,

Ashish Panwar, Jayashree

Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov 和 Ramachandran Ramjee. 2024. 使用 Sarathi-serve 调节 LLM 推理中的吞吐量 - 延迟权衡 . 载于 第 18 届 USENIX 操作系统设计与实现研讨会论文集 , 2024, 圣克拉拉 .

阿里巴巴 . 2024.[Qwen2.5 系列](#) . 访问日期 : 9 月 21 日 2024 年

人工分析. 2025. 人工分析: 人工智能模型性能基准测试。 . 访问

日期: 2025-05-17。 Barnard, Dom. 2022。 [平均](#)

[说话速度和每分钟单词数](#)。 访问日期: 2024 年 9

月 21 日。 Marc Brysbaert. 2019. 我们每分钟阅

读多少单词? 阅读速度的综述与元分析。 记忆与语

言杂志。 Daihang Chen, Yonghui Liu, Mingyi

Zhou, Yanjie Zhao, Haoyu Wang, Shuai Wang,

Xiao Chen, Tegawend  F Bissyand , Jacques

Klein, 和 Li Li. 2024. 面向移动端的大型语言模型:

初步路线图。 ACM 软件工程与方法学汇刊。

Lingjiao Chen, Matei Zaharia, 和 James Zou.

2023. Frugalgpt: 如何在降低成本的同时使用大

型语言模型并提高性能。 arXiv 预印本 arXiv:2305.

05176。 Cohere. 2024。 [命令模型](#)。 访问日期:

2024 年 9 月 21 日。 Tri Dao, Dan Fu, Stefano

Ermon, Atri Rudra, 和

阿里·迪巴, 维维克·夏尔马, 阿里·帕赞德, 哈梅德·皮尔西 - Avash 和 Luc Van Gool. 2017. 弱监督级联

卷积网络 . 见 IEEE 计算机视觉与模式识别会议论

文集 , 页 914–922. Dujian Ding, Ankur Mallick,

Chi Wang, Robert Sim, Subhabrata

Mukherjee, Victor Ruhle, Laks V. S.

Lakshmanan, 和 Ahmed Hassan Awadallah.

2024. [混合 LLM: 成本高效且质量感知的查询路](#)

[由](#) . Angela Fan, Mike Lewis, 和 Yann

Dauphin. 2018. 分层神经故事生成 . arXiv 预印

本 arXiv:1805.04833

. 访问日期: 2024 年 9 月 21 日 . 在 Gim,

Guojun Chen, Seung-seob Lee, Nikhil Sarda,

Anurag Khandelwal, 和 Lin Zhong. 2024.

Prompt cache: 低延迟推理的模块化注意力重用 .

机器学习与系统论文集

. 访问日期: 2024 年 9 月 21 日 .

Naman Goyal, Cynthia Gao, Vishrav

Chaudhar

[- 实践](#)。 在线; 访问时间: 2024 年 9 月 27

日 。 DeepSeek. 2024 年。 [Deepseek-v2.5](#)

[模型](#)。 访问时间: 2024 年 9 月 21 日。 DeepSeek.

2025 年。 Deepseek: 开源大语言模型。

. 访问时间: 2025 年 5 月 17 日。

Tim Dettmers, Mike Lewis, Younes Belkada,

和 Luke Zettlemoyer. 2022 年。 Gpt3.int8 ():

大规模变换器的 8 位矩阵乘法。 神经信息处理系统

进展, 35:30318–30332。

Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Jun-teng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kam-badur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthi, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Conguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Han-

cock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damraj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Habsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin,

李在元, 扬·格弗特, 雅娜·弗拉内斯, 贾森·朴, 杰伊·马哈德卡, 吉特·沙阿, 耶尔默·范德林德, 珍妮弗·比洛克, 珍妮·洪, 珍娅·李, 杰里米·傅, 建峰·池, 建宇·黄, 嘉文·刘, 杰·王, 杰曹·喻, 乔安娜·比顿, 乔·斯皮萨克, 张宗洙, 约瑟夫·罗卡, 约书亚·约翰斯顿, 约书亚·萨克斯, 贾腾·贾, 卡良·瓦苏登·阿尔瓦拉, 卡尔蒂克·普拉萨德, 卡尔蒂克耶·乌帕萨尼, 凯特·普拉维亚克, 李科, 肯尼思·希尔菲尔德, 凯文·斯通, 哈立德·艾尔-阿里尼, 克里蒂卡·艾耶, 克希蒂兹·马利克, 赵宽利, 库纳尔·巴拉, 库沙尔·拉科蒂亚, 劳伦·兰塔拉-耶里, 劳伦斯·范德马滕, 劳伦斯·陈, 梁坦, 莉兹·詹金斯, 路易斯·马丁, 洛维什·马丹, 鲁博·马洛, 卢卡斯·布莱赫, 卢卡斯·兰德扎特, 卢克·德·奥利维拉, 玛德琳·穆齐, 马赫什·帕苏普列蒂, 曼纳特·辛格, 马诺哈尔·帕卢里, 马尔钦·卡尔达斯, 玛丽亚·齐姆普凯利, 马修·奥尔德姆, 马修·丽塔, 玛雅·帕夫洛娃, 梅拉妮·坎巴杜尔, 迈克·刘易斯, 闵思, 米特什·库马尔·辛格, 莫娜·哈桑, 纳曼·戈亚尔, 纳尔杰斯·托拉比, 尼科莱·巴什利科夫, 尼科莱·博戈伊切夫, 尼拉德里·查特吉, 宁张

公鸡, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk

Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. **The llama 3 herd of models**.

Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, Andy Narayanan, Aonan Zhang, Bowen Zhang, Chen Chen, Chung-Cheng Chiu, David Qiu, et al. 2024. Apple intelligence foundation language models. *arXiv preprint arXiv:2407.21075*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Ashit Gupta, Anirudh Deodhar, Tathagata Mukherjee, and Venkataramana Runkana. 2022. Semi-supervised cascaded clustering for classification of noisy label data. *arXiv preprint arXiv:2205.02209*.

Mohammad Ashraful Hoque, Matti Siekkinen, Kashif Nizam Khan, Yu Xiao, and Sasu Tarkoma. 2015. Modeling, profiling, and debugging the energy consumption of mobile devices. *ACM Computing Surveys (CSUR)*, 48(3):1–40.

Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, et al. 2024. Mem-serve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*.

Hyperbolic. 2024. **Llama3-70b-instruct by hyperbolic**. Accessed on 21 Sep 2024.

Hyperbolic. 2025. Hyperbolic inference service: Host custom ai models and open-source llms. <https://app.hyperbolic.xyz>. Accessed: 2025-05-17.

Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2024. Fiddler: Cpu-gpu orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Hanchen Li, Yuhan Liu, Yihua Cheng, Siddhant Ray, Kuntai Du, and Junchen Jiang. 2024a. Eloquent: A more robust transmission scheme for llm token streaming. In *NAIC*.

Xiang Li, Zhenyan Lu, Dongqi Cai, Xiao Ma, and Mengwei Xu. 2024b. **Large language models on mobile devices: Measurements, analysis, and insights**. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*, EdgeFM ’24.

Bin Lin, Tao Peng, Chen Zhang, Minmin Sun, Lanbo Li, Hanyu Zhao, Wencong Xiao, Qi Xu, Xiafei Qiu, Shen Li, et al. 2024a. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669*.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Weiming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024b. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100.

Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*.

Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024a. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services. *arXiv preprint arXiv:2404.16283*.

Xiaozhe Liu, Ting Sun, Tianyang Xu, Feijie Wu, Cunxiang Wang, Xiaoqian Wang, and Jing Gao. 2024b. Shield: Evaluation and defense strategies for copyright compliance in llm text generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1640–1670.

Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024c. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. 2024d. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*.

盛鑫 Cindy Zha, Shishir Patil, Shiva Shankar, 张淑强, 张淑强, 王思农, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, 陈斯蒂芬, Steve Kehoe, Steve Satter eld, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, 赵松民, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, 李天河, 张天军, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, 李伟, 王文晨, 姜文文, Wes Bouaziz, Will Constable, 唐晓成, 吴晓建, 王晓岚, 吴希伦, 高鑫博, Yaniv Kleinman, 陈燕军, 胡烨, 贾烨, 叶祺, YendaLi, 张怡霖, 张颖, Yossi Adi, Youngjin Nam, 王宇, 赵宇, 郝宇辰, 钱云地, 李运鹭, 何宇滋, Zach Rait, Zachary DeVito, Zef Rosnbrick, 文昭多, Zh

权宇硕, 李卓翰, 庄思源, 英 Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, 和 Ion Stoica. 2023. 使用分页注意力的大型语言模型服务的高效内存管理。载于第 29 届 ACM SIGOPS 操作系统原则研讨会论文集。

李汉宸, 刘雨涵, 程一华, 西丹特·雷, 杜坤泰, 姜俊晨. 2024a. 《Eloquent: 一种更可靠的 LLM 令牌流传输方案》. 收录于 .

李翔、陆振岩、蔡东奇、马晓和孟 - 徐伟. 2024b. **移动设备上的大型语言模型: 测量、分析与见解**. 载于边缘与移动基础模型研讨会论文集

林斌, 彭涛, 张晨, 孙敏敏, 李蓝博, 赵涵宇, 肖文聪, 徐琦, 邱霞飞, 李深, 等. 2024a. In nite-llm: 基于 distattention 和分布式 kvcache 的长上下文高效 LLM 服务. arXiv 预印本 arXiv:2401.02669.

吉林, 贾明唐, 昊天唐, 上扬, 魏 - 陈明、王伟晨、肖光轩、党星宇、甘闯、韩松。2024b. AWQ: 面向设备端大语言模型压缩与加速的激活感知权重量化。机器学习与系统会议论文集, 6:87–100.

刘昊, 马特·扎哈里亚, 和 彼得·阿贝尔. 2023. 基于块的变换器的环状注意力用于近无限上下文。arXiv 预印本 arXiv:2310.01889。

刘嘉晨、吴志宇、郑在元、范赖明振, 李明镇, 和莫沙拉夫·乔杜里。2024a. 《Andes: 定义和提升基于大语言模型的文本流服务的体验质量》。arXiv 预印本 arXiv:2404.16283。

刘晓泽, 孙婷, 徐天阳, 吴飞杰, 存 x-王江、王晓倩和高晶。2024b. 《Shield: 大语言模型文本生成中版权合规的评估与防御策略》。发表于《2024 年自然语言处理实证方法会议论文集》, 第 1640–1670 页。

刘宇涵, 李汉宸, 程一华, 西丹特·雷, 黄宇阳, 张启正, 杜坤泰, 姚佳怡, 卢山, Ganesh Ananthanarayanan 等. 2024c. Cachegen: KV 缓存压缩和流式处理用于快速大语言模型服务. 载于 ACM SIGCOMM 2024 会议论文集, 页 38–56.

刘泽春, 赵长生, Forrest Iandola, 陈 Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi 等人. 2024d. Mobilellm: 优化亚十亿参数语言模型以用于设备端应用。arXiv 预印本 arXiv:2402.14905。

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2024e. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36.

Teemu Mäki-Patola and Perttu Härmäläinen. 2004. Latency tolerance for gesture controlled continuous sound instrument without tactile feedback. In *International Computer Music Conference*. Citeseer.

Microsoft Azure. 2025. Azure ai foundry: Model catalog and ai services. <https://azure.microsoft.com/en-us/solutions/ai-foundry>. Accessed: 2025-05-17.

MLC team. 2023. [MLC-LLM](#).

Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. 2016. Abstractive text summarization using sequence-to-sequence rnns and beyond. *arXiv preprint arXiv:1602.06023*.

Chaoyue Niu, Yucheng Ding, Junhui Lu, Zhengxiang Huang, Hang Zeng, Yutong Dai, Xuezhen Tu, Chengfei Lv, Fan Wu, and Guihai Chen. 2025. Collaborative learning of on-device small model and cloud-based large model: Advances and future directions. *arXiv preprint arXiv:2504.15300*.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2024. [Routellm: Learning to route llms with preference data](#).

OpenAI. 2022. [tiktoken](#). Accessed on 21 Sep 2024.

OpenAI. 2024. [Gpt-4o mini model](#). Accessed on 21 Sep 2024.

Tara Parachuk. 2022. [Speaking rates comparison table](#). Accessed on 21 Sep 2024.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. [Splitwise: Efficient generative llm inference using phase splitting](#). In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.

Aleksandar Petrov, Emanuele La Malfa, Philip H.S. Torr, and Adel Bibi. 2024. Language model tokenizers introduce unfairness between languages. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. 2025. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–624.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: Kimi’s kvcache-centric architecture for llm serving. *arXiv preprint arXiv:2407.00079*.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. Powerinfer: Fast large language model serving with a consumer-grade gpu. *arXiv preprint arXiv:2312.12456*.

Ting Sun, Penghan Wang, and Fan Lai. 2025. [Hygen: Efficient llm serving via elastic online-offline request co-location](#).

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

NLLB Team, Marta R Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Hefernan, Elahe Kalbassi, Janice Lam, Daniel Licht, et al. 2022. No language left behind: Scaling human-centered machine translation (2022). *URL https://arxiv.org/abs/2207.04672*.

Together AI. 2025. Together ai: High-performance inference for open-source llms. <https://www.together.ai>. Accessed: 2025-05-17.

Jakub Žádník, Markku Mäkitalo, Jarno Vanne, and Pekka Jämskeläinen. 2022. [Image and video coding techniques for ultra-low latency](#). *ACM Comput. Surv.*

Zijie J. Wang, Evan Montoya, David Munechika, Haoyang Yang, Benjamin Hoover, and Duen Horng Chau. 2022. [DiffusionDB: A large-scale prompt gallery dataset for text-to-image generative models](#). *arXiv:2210.14896 [cs]*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR.

Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2023. Llmcad: Fast and scalable on-device large language model inference. *arXiv preprint arXiv:2309.04255*.

刘子昌、阿迪提亚·德赛、廖方硕、王维涛、谢维克多、徐兆卓、阿纳斯塔西奥斯·基里利迪斯、安舒马利·什里瓦斯塔瓦。2024 年 e. Scissorhands: 利用重要性持久性假设进行大语言模型键值缓存压缩的验证时间。神经信息处理系统进展, 36.

Teemu Mäki-Patola 和 Perttu Härmäläinen. 2004. 无触觉反馈的手势控制连续音响乐器的延迟容忍度。在国际计算机音乐会议。Citeseer.

微软 Azure. 2025. Azure AI 工厂：模型目录和 AI 服务 .<https://azure.microsoft.com/zh-cn/solutions/ai-foundry>. 访问日期：2025-05-17.

MLC 团队 . 2023.

Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang 等 . 2016. 使用序列到序列 RNN 及其扩展的抽象文本摘要。arXiv 预印本 arXiv:1602.06023.

牛超越, 丁宇成, 陆俊辉, 黄正祥, 曾航, 戴宇彤, 涂雪珍, 吕承飞, 吴凡, 陈贵海 . 2025. 设备端小模型与云端大模型的协作学习：进展与未来方向 . arXiv 预印本 arXiv:2504.15300.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E.Gonzalez, M Waleed Kadous, 和 Ion Stoica. 2024.[Routellm：使用偏好数据学习路由大型语言模型](#)

访问日期：2024 年 9 月 21 日。

[Gpt-4o mini 模型](#) . 访问日期：2024 年 9 月 21 日 .

[说话速度比较表](#)。访问于 2024 年 9 月 21 日。

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, 和 Ricardo Bianchini. 2024. [Splitwise: 使用阶段拆分的高效生成式 LLM 推理](#)。发表于 2024 ACM/IEEE 第 51 届年度国际计算机体系结构研讨会 (ISCA)。

Aleksandar Petrov, Emanuele La Malfa, Philip H.S. Torr, 和 Adel Bibi. 2024. 语言模型的分词器在语言之间引入不公平。载于第 37 届国际神经信息处理系统会议论文集。

Long Phan、Alice Gatti、Ziwen Han、Nathaniel Li、Josephina Hu、Hugh Zhang、Chen Bo Calvin Zhang、Mohamed Shaaban、John Ling、Sean Shi 等。2025。人类的最后一次考试。arXiv 预印本 arXiv:2501.14249。

赖纳·波普, 绍尔托·道格拉斯, 阿坎克沙·乔德里, Jacob Devlin, James Bradbury, Jonathan Heek, Xiao Kefan, Shivani Agrawal, 和 Jeff Dean. 2023. 高效扩展变压器推理。机器学习与系统会议论文集, 5:606–624.

秦若宇, 李哲明, 何维然, 张明兴, 吴永伟, 郑伟民, 徐欣然 . 2024. Mooncake: Kimi 的以 kvcache 为中心的 LLM 服务架构 . arXiv 预印本 arXiv:2407.00079.

穆罕默德·肖耶比, 莫斯塔法·帕特瓦里, 劳尔·普里, Patrick LeGresley, Jared Casper, 和 Bryan Catanzaro. 2019. Megatron-lm: 使用模型并行训练数十亿参数语言模型。arXiv 预印本 arXiv:1909.08053。

宋一心, 米泽宇, 谢昊通, 陈海波 . 2023. Powerinfer: 使用消费级 GPU 提供快速大型语言模型服务。arXiv 预印本 arXiv:2312.12456。

孙亭, 王鹏涵, 赖凡 . 2025. [通过弹性在线请求共置实现高效的 LLM 服务](#)

罗汉·塔奥里, 伊山·古拉贾尼, 张天意, 扬 Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, 和 Tatsunori B. Hashimoto. 2023. Stanford alpaca: 一个遵循指令的羊驼模型 .

NLLB 团队, Marta R Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Hea eld, Kevin Hefernan, Elahe Kalbassi, Janice Lam, Daniel Licht 等人 . 2022. 没有语言被落下: 扩展以人为中心的机器翻译 (2022))。网址 <https://arxiv.org/abs/2207.04672>。

Together AI. 2025. Together AI: 高性能开源大语言模型的推理。 . 访问日期：2025-05-17。

Jakub ádník、Markku Mäkitalo、Jarno Vanne, 以及 [超低延迟的图像和视频编码技术](#)。ACM 计算机调查

王子杰, 埃文·蒙托亚, 大卫·穆内奇卡, Haoyang Yang, Benjamin Hoover, 和 DuenHorng Chau. 2022. [DiffusionDB: 用于文本到图像生成模型的大规模提示图库数据集](#)

肖广轩, 季林, 米卡埃尔·塞兹内克, 吴浩, Julien Demouth 和 Song Han. 2023. Smoothquant: 用于大型语言模型的精确且高效的训练后量化。在国际机器学习会议, 第 38087–38099 页。PMLR。

徐大亮, 尹王松, 金鑫, 张颖, 魏世云, 徐梦炜, 刘轩哲 . 2023. Llmcad: 快速且可扩展的设备端大语言模型推理 . arXiv 预印本 arXiv:2309.04255.

Nan Xue, Yaping Sun, Zhiyong Chen, Meixia Tao, Xiaodong Xu, Liang Qian, Shuguang Cui, and Ping Zhang. 2024a. Wdmoe: Wireless distributed large language models with mixture of experts. *arXiv preprint arXiv:2405.03131*.

Zhenliang Xue, Yixin Song, Zeyu Mi, Le Chen, Yubin Xia, and Haibo Chen. 2024b. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*.

Zheming Yang, Yuanhao Yang, Chang Zhao, Qi Guo, Wenkai He, and Wen Ji. 2024. Perllm: Personalized inference scheduling with edge-cloud collaboration for diverse llm services. *arXiv preprint arXiv:2405.14636*.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. [Flashinfer: Efficient and customizable attention engine for llm inference serving](#). *arXiv preprint arXiv:2501.01005*.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soo-jeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*.

Mingjin Zhang, Jiannong Cao, Xiaoming Shen, and Zeyang Cui. 2024. Edgeshard: Efficient llm inference via collaborative edge computing. *arXiv preprint arXiv:2405.14371*.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*.

A Additional Related Work

General LLM Inference. LLMs generate text responses auto-regressively, producing one token at a time based on preceding tokens. The process consists of two stages that can potentially be executed on different endpoints: (i) *Prefill stage*: The model processes the input text (prompt), calculates and stores intermediate model states—i.e., the key and value cache (KV cache) of tokens—to generate the first token. A token represents a word or part of a word that the model can interpret. Once the first token is generated, it is appended to the end of the prompt and the generation process moves on to the (ii) *Decode stage*: The model processes the updated prompt (including previously generated tokens) to generate the next token. The decode stage continues until a stopping condition is met (e.g., reaching

an end-of-sequence token or the maximum generation length).

On-Server LLM Serving. Existing works have focused on GPU kernel optimization (Dao et al., 2022; Ye et al., 2025), KV-cache management (Lin et al., 2024a; Liu et al., 2024e; Qin et al., 2024), model parallelism (Shoeybi et al., 2019; Pope et al., 2023; Liu et al., 2023), quantization (Xiao et al., 2023; Lin et al., 2024b; Dettmers et al., 2022), and scheduling (Yu et al., 2022; Kwon et al., 2023; Agrawal et al., 2024; Sun et al., 2025). For example, Orca (Yu et al., 2022) introduced continuous batching to improve serving throughput, while vLLM (Kwon et al., 2023) developed PagedAttention to reduce LLM memory restraint. Sarathi (Agrawal et al., 2024) implemented chunked prefill to mitigate inter-request interference within batches. Andes (Liu et al., 2024a) addresses QoE for individual requests from the server side but lacks awareness of network jitter and device potential. These server-side advancements complement DiSCo’s design.

On-Device LLMs. Google’s Gemini Nano (Google, 2024) and Apple’s Apple Intelligence (Gunter et al., 2024) have been integrated into Android OS and iOS devices, respectively. MLC-LLM (MLC team, 2023) and llama.cpp (Gerganov, 2024) efficiently deploy various LLMs on devices. PowerInfer (Song et al., 2023) and PowerInfer-2 (Xue et al., 2024b) optimize on-device LLM inference by leveraging sparsity in model activations. DiSCo acts as a middle layer to schedule and migrate response generation between servers and devices.

B Cold Start Evaluation

This section presents cold start performance measurements for the Qwen-2.5 model series across different hardware configurations. The experiments were conducted on two platforms: Windows 10 with NVIDIA RTX 3060 12GB and Linux with NVIDIA A40 48GB. A fixed prompt "How to use GitHub?" was used throughout all experiments. We measured two critical metrics: model loading time and TTFT for Qwen-2.5 models ranging from 0.5B to 7B parameters, all using FP16 precision. The experimental setup consisted of 10 measurement runs, with two additional warmup runs to ensure measurement stability. It is worth noting that such warmups can potentially mask the true gap between model loading and prompt prefill time due to var-

南雪, 孙亚平, 陈志勇, 陶美霞, 徐小东, 钱亮, 崔曙光, 张平. 2024a. Wdmoe: 具有专家混合的无线分布式大语言模型. arXiv 预印本 arXiv:2405.03131.

薛振良, 宋一心, 米泽宇, 陈乐, 夏宇彬, 陈海波. 2024b. Powerinfer-2: 在智能手机上进行大规模语言模型快速推理. arXiv 预印本 arXiv:2406.06282.

Zheming Yang, Yuanhao Yang, Chang Zhao, Qi Guo, Wenkai He 和 Wen Ji. 2024. Perllm: 面向多样化 LLM 服务的个性化推理调度与边缘 - 云协作. arXiv 预印本 arXiv:2405.14636.

叶子昊, 陈乐群, 赖睿航, 林武伟, 张宜能, 王佳慧, 陈天琪, 巴里斯·卡西克奇, 维诺德·格罗弗, 阿尔温德·克里希纳穆尔蒂, 路易斯·塞泽. 2025. [Flashinfer: 高效且可定制的注意力引擎用于 LLM 推理服务](#) arXiv 预印本 arXiv:2501.01005

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soo-jeong Kim, 和 Byung-Gon Chun. 2022. Orca: 一个用于基于 Transformer 的生成模型的分布式服务系统. 发表于 第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22).

张明进, 曹建农, 沈晓明, 崔泽阳. 2024. Edgeshard: 通过协作边缘计算提高大型语言模型推理效率. arXiv 预印本 arXiv:2405.14371.

钟银民, 刘胜宇, 陈俊达, 胡建波, 朱一波, 刘轩哲, 金鑫, 张浩. 2024. DistServe: 为优化有效吞吐量的大型语言模型服务进行预 LL 拆分和解码. 在第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24) 上.

附加相关工作

通用大型语言模型推理. 大型语言模型 (LLMs) 以自回归方式生成文本响应, 每次根据前面的标记生成一个标记。该过程包括两个可能在不同端点执行的阶段: (i) 预处理阶段: 模型处理输入文本 (提示), 计算并存储中间模型状态 —— 即标记的键值缓存 (KV 缓存) —— 以生成第一个标记。一个标记代表模型能够解释的一个词或词的一部分。一旦生成第一个标记, 它会被追加到提示的末尾, 生成过程进入 (ii) 解码阶段: 模型处理更新后的提示 (包括之前生成的标记) 以生成下一个标记。解码阶段将持续, 直到满足停止条件 (例如, 达到

一个序列结束标记或最大生成长度。

服务器端大语言模型服务。现有研究主要集中在 GPU 内核优化 (Dao 等, 2022 ; Ye 等, 2025)、KV 缓存管理 (Lin 等, 2024a ; Liu 等, 2024e ; Qin 等, 2024)、模型并行 (Shoeybi 等, 2019 ; Pope 等, 2023 ; Liu 等, 2023)、量化 (Xiao 等, 2023 ; Lin 等, 2024b ; Dettmers 等, 2022) 以及调度 (Yu 等, 2022 ; Kwon 等, 2023 ; Agrawal 等, 2024 ; Sun 等, 2025)。例如, Orca (Yu 等, 2022) 引入了连续批处理以提高服务吞吐量, 而 vLLM (Kwon 等, 2023) 开发了 PagedAttention 以减少 LLM 的内存限制。Sarathi (Agrawal 等, 2024) 实现了分块预填充以缓解批处理内的请求间干扰。Andes (Liu 等, 2024a) 从服务器端关注单个请求的 QoE, 但缺乏对网络抖动和设备潜力的感知。这些服务器端的进展补充了 DiSCo 的设计。

设备端大语言模型 (LLM)。 谷歌的 Gemini Nano (,) 和苹果的 Apple Intelligence (Gunter 等,) 已分别集成到 Android OS 和 iOS 设备中。MLC-LLM (MLC 团队,) 和 llama.cpp (,) 高效地在设备上部署各种 LLM。PowerInfer (Song 等,) 和 PowerInfer-2 (Xue 等,) 通过利用模型激活中的稀疏性优化设备端 LLM 推理。 作为中间层, 在服务器和设备之间调度和迁移响应生成。

B ColdStart 评估

本节展示了 Qwen-2.5 模型系列在不同硬件配置下的冷启动性能测量。实验在两个平台上进行: Windows 10 配置 NVIDIA RTX 3060 12GB, 以及 Linux 配置 NVIDIA A40 48GB。在所有实验中均使用固定提示 “如何使用 GitHub? ”。我们测量了两个关键指标: 模型加载时间和 Qwen-2.5 模型 (参数量从 0.5B 到 7B) 在 FP16 精度下的 TTFT。实验设置包括 10 次测量运行, 并增加了两次预热运行以确保测量稳定性。值得注意的是, 这类预热可能会掩盖模型加载与提示预填充时间之间的真实差距, 因为

ious optimizations, including OS page cache. To maintain authentic cold start conditions, we explicitly cleared the CUDA cache and performed garbage collection before each run.

The results revealed several significant patterns. On the RTX 3060, the loading time exhibits an approximately linear increase with model size, ranging from 1.29s for the 0.5B model to 4.45s for the 3B model. While TTFT follows a similar trend, the processing time is substantially lower, ranging from 0.051s to 0.145s. On the A40 GPU, despite observing longer loading times, TTFT is significantly reduced across all models, maintaining a remarkably consistent value regardless of model size. These findings indicate that while model loading remains more resource-intensive on our Linux setup, the inference performance benefits substantially from the A40’s superior computational capabilities.

Metric	0.5B	1.5B	3B	7B
Windows 10 (NVIDIA RTX 3060 12GB)				
Load Time (s)	1.29	2.48	4.45	-
TTFT (s)	0.051	0.105	0.145	-
Linux (NVIDIA A40 48GB)				
Load Time (s)	1.53	3.12	5.72	13.43
TTFT (s)	0.025	0.026	0.033	0.033

Table 4: Model loading time during cold start can significantly slow down TTFT. Average Qwen-2.5 model performance over 10 runs. The 7B model exceeds the memory capacity of the RTX 3060 and thus cannot be evaluated.

C Prediction-based Model Selection

This section provides a comparative analysis of several TTFT prediction methods. For selecting the endpoint with a lower TTFT for each request, TTFT prediction is imperative. For on-device inference, TTFT prediction is straightforward, as TTFT exhibits a linear relationship with prompt length. Conversely, on-server TTFT is characterized by high variability, posing challenges for prediction. Moreover, the prediction method itself must be computationally efficient, as its overhead also contributes to end-to-end TTFT.

Table 5 presents a comparative analysis of four common lightweight time-series-based prediction methods applied to traces collected from three prevalent LLM services. Our correlation analysis (Table 1) reveals no significant correlation between prompt length and TTFT; thus, prompt length is

Model	MAPE(%)	MAE(s)
Command		
Moving Average	39.40	0.0899
ExponentialSmoothing	53.51	0.1047
Random Forest	39.33	0.0966
XGBoost	35.43	0.0905
DeepSeek-V2.5		
Moving Average	27.80	0.3959
ExponentialSmoothing	27.39	0.3771
Random Forest	32.97	0.4745
XGBoost	27.51	0.4001
GPT-4o-mini		
Moving Average	24.55	0.0995
ExponentialSmoothing	20.88	0.0844
Random Forest	28.68	0.1128
XGBoost	24.83	0.0997
LLaMA-3-70b-Instruct		
Moving Average	42.18	0.3312
ExponentialSmoothing	40.27	0.3154
Random Forest	49.67	0.3875
XGBoost	43.94	0.3451

Table 5: Comparative analysis of Moving Average, Exponential Smoothing, Random Forest, and XGBoost prediction models across Command, DeepSeek, GPT, and LLaMA model traces. Metrics include Mean Absolute Percentage Error (MAPE) and Mean Absolute Error (MAE).

omitted as a feature in these prediction methods. We demonstrate that none of these methods offers sufficient accuracy for TTFT prediction.

To address potential concerns about the robustness of our server-side TTFT prediction under varying workload conditions, we emphasize that our distribution-based prediction model is specifically designed to handle workload bursts and dynamic patterns. The statistical distribution we maintain inherently captures these patterns, allowing the model to automatically adapt to changing workloads based on historical data. Furthermore, our system architecture supports dynamic distribution updates through a lightweight server API, which can efficiently transmit real-time server workload information using just tens or hundreds of numbers. This design ensures that our prediction model remains responsive and accurate even during significant load spikes or ephemeral workloads.

D Response Quality

This section examines the quality of responses generated by *DiSCo*, with a particular focus on quality preservation during endpoint transitions. We first

各种优化，包括操作系统页面缓存。为了保持真实的冷启动条件，我们在每次运行前明确清除了 CUDA 缓存并进行了垃圾回收。

结果显示了几个显著的模式。在 RTX 3060 上，加载时间随着模型大小呈近似线性增加，从 0.5B 模型的 1.29 秒到 3B 模型的 4.45 秒。虽然 TTFT 呈现类似的趋势，但处理时间明显较低，范围从 0.051 秒到 0.145 秒。在 A40 GPU 上，尽管观察到加载时间更长，但 TTFT 在所有模型上均显著降低，并且无论模型大小如何，都保持了非常一致的数值。这些发现表明，虽然在我们的 Linux 环境下模型加载仍然更耗资源，但推理性能却显著受益于 A40 卓越的计算能力。

Metric	0.5B	1.5B	3B	7B
Windows 10 (NVIDIA RTX 3060 12GB)				
Load Time (s)	1.29	2.48	4.45	-
TTFT (s)	0.051	0.105	0.145	-
Linux (NVIDIA A40 48GB)				
Load Time (s)	1.53	3.12	5.72	13.43
TTFT (s)	0.025	0.026	0.033	0.033

表 4：冷启动期间的模型加载时间可能会显著减慢 TTFT。在 10 次运行中 Qwen-2.5 模型的平均性能。7B 模型超过了 RTX 3060 的内存容量，因此无法进行评估。

C 基于预测的模型选择

本节提供了对几种 TTFT 预测方法的比较分析。为了为每个请求选择 TTFT 较低的端点，TTFT 预测是必不可少的。对于设备端推理来说，TTFT 预测相对简单，因为 TTFT 与提示长度呈线性关系。相反，服务器端 TTFT 具有高度可变性，这给预测带来了挑战。此外，预测方法本身必须具备计算高效性，因为其开销也会影响端到端的 TTFT。

表 展示了四种常见轻量级基于时间序列的预测方法的比较分析，这些方法应用于从三种流行的 LLM 服务收集的追踪数据。我们的相关性分析（表 ）显示提示长度与 TTFT 之间没有显著相关性；因此，提示长度是

Model	MAPE(%)	MAE(s)
Command		
Moving Average	39.40	0.0899
ExponentialSmoothing	53.51	0.1047
Random Forest	39.33	0.0966
XGBoost	35.43	0.0905
DeepSeek-V2.5		
Moving Average	27.80	0.3959
ExponentialSmoothing	27.39	0.3771
Random Forest	32.97	0.4745
XGBoost	27.51	0.4001
GPT-4o-mini		
Moving Average	24.55	0.0995
ExponentialSmoothing	20.88	0.0844
Random Forest	28.68	0.1128
XGBoost	24.83	0.0997
LLaMA-3-70b-Instruct		
Moving Average	42.18	0.3312
ExponentialSmoothing	40.27	0.3154
Random Forest	49.67	0.3875
XGBoost	43.94	0.3451

Table 5: Comparative analysis of Moving Average, Exponential Smoothing, Random Forest, and XGBoost prediction models across Command, DeepSeek, GPT, and LLaMA model traces. Metrics include Mean Absolute Percentage Error (MAPE) and Mean Absolute Error (MAE).

在这些预测方法中被作为一个特征省略。我们证明了这些方法都没有提供足够的 TTFT 预测精度。

为了应对关于我们服务器端 TTFT 预测在不同工作负载条件下稳健性可能的担忧，我们强调，我们的基于分布的预测模型是专门设计来处理工作负载突发和动态模式的。我们维护的统计分布本质上捕捉了这些模式，使模型能够根据历史数据自动适应变化的工作负载。此外，我们的系统架构支持通过轻量级服务器 API 动态更新分布，该 API 可以仅使用几十或几百个数字高效传输实时服务器工作负载信息。这一设计确保了即使在显著的负载峰值或短暂工作负载期间，我们的预测模型仍然保持响应迅速且准确。

D 响应质量

本节考察了 生成的响应质量，特别关注在端点转换期间的质量保持。我们首先

establish bounds on generation quality, then present our evaluation methodology, and finally demonstrate through extensive experiments that *DiSCo* maintains consistent quality across different model configurations and tasks.

D.1 Quality Bounds

A critical aspect of *DiSCo* is maintaining generation quality during endpoint transitions. We employ a systematic approach to quality preservation (Diba et al., 2017; Gupta et al., 2022; Chen et al., 2023). Specifically, for endpoints A and B with quality metrics Q_A and Q_B (measured by LLM scores or ROUGE scores), we find that any migrated sequence M with quality Q_M satisfies:

$$\min(Q_A, Q_B) \leq Q_M \leq \max(Q_A, Q_B) \quad (7)$$

This bound ensures that migration does not degrade quality beyond the capabilities of individual endpoints.

D.2 Evaluation Methodology

Evaluation Framework We establish a comprehensive assessment framework encompassing both automated metrics and LLM-based evaluation. Our framework evaluates two distinct tasks:

- **Translation Quality:** We assess Chinese-to-English translation on 500 data items from Flores_zho_Hans-eng_Latn dataset (Team et al., 2022; Goyal et al., 2022) using the ROUGE-1 metric.
- **Instruction Following:** We evaluate 500 data items from the Alpaca dataset (Taori et al., 2023) using our structured prompt template, with quality assessment performed by multiple LLM judges: Gemini1.5-pro, GPT-4o, and QWen2.5-72b-instruct.
- **Text Summarization:** We evaluate 100 data items from the CNN/DM (Nallapati et al., 2016) using our structured prompt template, with quality assessment performed by DeepSeek-V3-0324.
- **Story Writing:** We evaluate 100 data items from the WritingPrompts (Fan et al., 2018) using our structured prompt template, with quality assessment performed by DeepSeek-V3-0324.

These two tasks are popular on end-user devices. Understandably, for complex tasks such as advanced math reasoning, we notice DisCo can lead to accuracy drops compared to the on-server model due to the limited capability of the on-device models, yet still achieves better performance than the on-device counterpart.

Experimental Setup We configure our experiments with:

- A fixed maximum generation length of 256 tokens
- First endpoint’s maximum generation length varied through different (e.g., [0, 4, 16, 64, 256] tokens).
- Four model combinations: 0.5B-7B, 3B-7B, 7B-0.5B, and 7B-3B (prefix and suffix denote the model sizes of first and second endpoints, respectively)

The generation transitions to the second endpoint when the first endpoint reaches its length limit without producing an end-of-generation token, creating natural boundary conditions for analysis.

For instruction-following tasks, we employ the following structured evaluation template:

JUDGE_PROMPT = """Strictly evaluate the quality of the following answer on a scale of 1-10 (1 being the worst, 10 being the best). First briefly point out the problem of the answer, then give a total rating in the following format.

Question: {question}

Answer: {answer}

Evaluation: (your rationale for the rating, as a brief text)

Total rating: (your rating, as a number between 1 and 10)
"""

D.3 Results and Analysis

D.3.1 Quality Metrics

Our comprehensive evaluation reveals that the combined sequences’ quality consistently remains bounded between individual model performance levels.

E Experiment Settings for End-to-end Cost

For on-device LLMs, we quantify cost using FLOPs (floating-point operations). For on-server

建立边界生成质量，然后介绍我们的评估方法，最后通过大量实验演示 在不同模型配置和任务中保持一致的质量。

D.1 质量界限

DiSCo 的一个关键方面是在端点过渡期间保持生成质量。我们采用系统的方法来保持质量 (Diba 等人, ; Gupta 等人, ; Chen 等人,)。具体来说，对于端点 A 和 B 及其质量指标 和 (由 LLM 分数或 ROUGE 分数测量)，我们发现任何迁移序列 M 的质量 满足：

$$\min(Q_A, Q_B) \leq Q_M \leq \max(Q_A, Q_B) \quad (7)$$

此界限确保迁移不会使质量下降到超过各个端点能力的程度。

D.2 评估方法

评估框架我们建立了一个涵盖自动化指标和基于大型语言模型（LLM）评估的综合评估框架。我们的框架评估两项不同的任务：

翻译质量：我们使用 ROUGE-1 指标评估了来自 Flores_zho_Hans-eng_Latn 数据集的 500 个中译英条目（Team 等 Goyal 等 ）。

指令遵循：我们评估 500 个数据使用我们的结构化提示模板从 Alpaca 数据集提取的项目（Taori 等人, ），质量评估由多位 LLM 评审执行：Gemini1.5-pro、GPT-4o 和 QWen2.5-72b-instruct。

文本摘要：我们评估 100 来自 CNN/DM 的数据项（Nallapati 等人, ）使用我们的结构化提示模板，质量评估由 DeepSeek-V3-0324 执行。

故事写作：我们评估 100 个数据项来自 WritingPrompts (Fan 等人), 使用我们的结构化提示模板，并由 DeepSeek-V3-0324 执行质量评估。

这两项任务在终端用户设备上很受欢迎。可以理解的是，对于像高级数学推理这样复杂的任务，我们注意到由于设备上模型能力有限，DisCo 相较于服务器端模型可能会导致准确率下降，但仍然比设备端的对应模型表现更好。

实验设置我们将实验配置如下：

固定的最大生成长度为 256 个标记

第一个端点的最大生成长度在不同（例如，令牌）之间有所变化。

四种模型组合：0.5B-7B、3B-7B、7B-0.5B 和 7B-3B（前缀和后缀分别表示第一个和第二个端点的模型大小）

当第一个端点在没有生成结束生成标记的情况下达到其长度限制时，生成会过渡到第二个端点，为分析创造自然的边界条件。

对于遵循指令的任务，我们采用以下结构化评估模板：

JUDGE_PROMPT 严格评估以下答案的质量，评分范围为 1-10（1 最差，10 最好）。首先简要指出答案的问题，然后按以下格式给出总体评分。

问题: {question}

答案: {answer}

评价: （您给出评分的理由，简要说明）

总评分: （您的评分，介于 1 到 10 之间的数字）

D.3 结果与分析

D.3.1 质量指标

我们的综合评估显示，组合序列的质量始终保持在各个单独模型性能水平之间。

端到端成本的实验设置

对于本地设备上的大型语言模型，我们使用浮点运算（FLOPs）来量化成本。对于服务器端

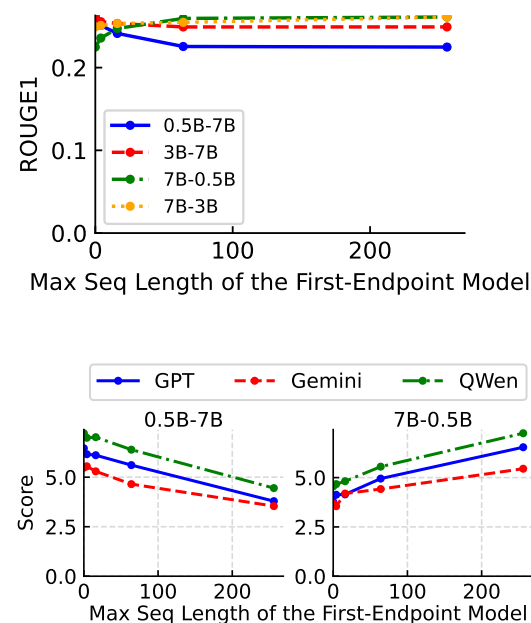


Figure 10: Quality evaluation results of *DiSCo*. The top figure shows translation quality evaluation using ROUGE-1 scores, demonstrating that *DiSCo* consistently achieves higher quality than the on-device baseline. The bottom figure presents evaluation scores from different LLM judges on instruction-following capabilities, where each subplot represents a different model pair comparison with varied first-endpoint model's maximum sequence length. The consistent patterns across different LLM judges demonstrate the robustness of our evaluation framework.

LLM services, we use their respective pricing rates at the time of experimentation. We set the energy-to-monetary conversion ratio (*energy_to_money*) to 0.3 \$ per million FLOPs for server-constrained experiments and 5 \$ per million FLOPs for device-constrained experiments. To establish a comprehensive cost model that enables direct comparison between device and server computation costs, we analyze both the computational complexity of on-device models through detailed FLOPs calculations (Section E.2) and the pricing structures of commercial

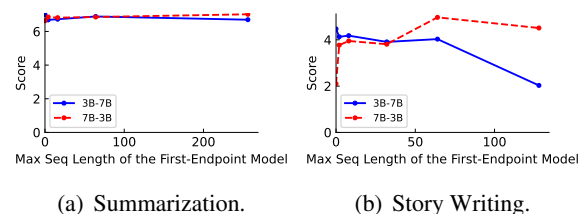


Figure 11: Quality evaluation results with Summarization and Story Writing.

LLM services (Section E.3). The generation length limit is set to 128.

E.1 Theoretical Cost Modeling

Our unified cost model combines heterogeneous costs (server monetary and device energy) through a dynamic exchange rate λ , which users adjust based on preferences (e.g., battery level, server budget). We formalize this as:

$$\text{Total Cost} = \underbrace{c_s^p \cdot l_s + c_s^d \cdot l_s}_{\text{Server Cost}} + \lambda \cdot \underbrace{(c_d^p \cdot l_d + c_d^d \cdot l_d)}_{\text{Device Cost}} \quad (8)$$

where l_s, l_d denote tokens processed on the server and the device, respectively. The optimization goal is to minimize $\mathbb{E}[\text{Total Cost}]$ while satisfying TTFT/TBT constraints. This formulation allows us to:

- Balance between server monetary costs and device energy consumption
- Adapt to user preferences through the exchange rate λ
- Account for different costs in prefill (c^p) and decode (c^d) phases
- Optimize token allocation between server and device execution

The model's flexibility enables it to handle both device-constrained scenarios (where energy consumption is the primary bottleneck) and server-constrained scenarios (where API monetary costs dominate).

While we acknowledge that real-world device energy consumption can be influenced by factors such as battery levels, thermal throttling, and concurrency, we use the linear FLOPs-based measure as a generalizable proxy metric for two key reasons: (1) to cover energy consumption across diverse devices (phones, laptops, and edge servers) and models with varying architectures and quantizations that may exhibit different consumption patterns, and (2) to target short conversational tasks where on-device LLMs demonstrate sufficient efficiency and effectiveness. Importantly, since our dispatching and migration algorithms are both length-threshold-based, they can adapt to any complex energy consumption patterns as long as these patterns are predictable and guaranteed to consume more energy when prefilling or decoding additional tokens.

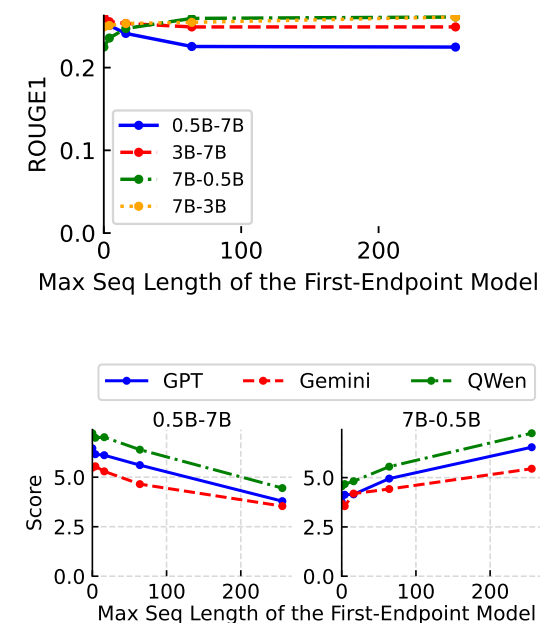


图 10: 的质量评估结果。上图显示了使用 ROUGE-1 分数进行的翻译质量评估, 表明 始终比设备上的基线模型实现更高的质量。下图展示了来自不同大型语言模型 (LLM) 评审在指令遵循能力方面的评估分数, 其中每个子图代表不同模型对比的结果, 并且首端模型的最大序列长度各不相同。不同 LLM 评审之间的一致模式展示了我们评估框架的稳健性。

LLM 服务, 我们使用它们在实验时的各自定价。我们将能量到货币的转换比率 () 设置为每百万 FLOPs 0.3 美元, 用于服务器受限实验, 而设备受限实验则为每百万 FLOPs 5 美元。为了建立一个能够直接比较设备与服务器计算成本的综合成本模型, 我们通过详细的 FLOPs 计算分析了设备端模型的计算复杂度 (章节) 以及商业定价结构

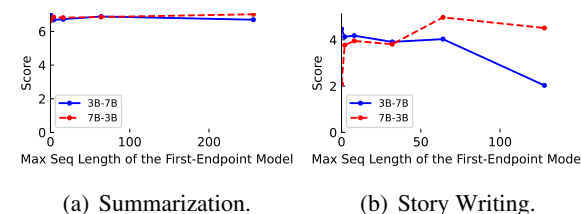


图 11: 使用摘要和故事写作的质量评估结果。

cial LLM 服务 (第 节)。生成长度限制为 128。

E.1 理论成本建模

我们的统一成本模型通过动态汇率 结合了异构成本 (服务器资金成本和设备能耗), 用户根据偏好 (例如电池电量、服务器预算) 进行调整。我们将其形式化如下:

$$\text{Total Cost} = \underbrace{c_s^p \cdot l_s + c_s^d \cdot l_s}_{\text{Server Cost}} + \lambda \cdot \underbrace{(c_d^p \cdot l_d + c_d^d \cdot l_d)}_{\text{Device Cost}} \quad (8)$$

其中 分别表示在服务器和设备上处理的标记。优化目标是在满足 TTFT/TBT 约束的同时最小化 E 总成本。此公式允许我们:

服务器货币成本与设备能耗之间的平衡

通过汇率适应用户偏好

在预 LL 阶段 () 和解码阶段 () 考虑不同的成本

优化服务器与设备执行之间的令牌分配

该模型的灵活性使其能够处理设备受限的场景 (能耗是主要瓶颈) 和服务器受限的场景 (API 金钱成本占主导)。

虽然我们承认现实世界中设备的能耗可能会受到电池电量、热限制和并发等因素的影响, 但我们使用基于线性 FLOPs 的度量作为可概率化的代理指标有两个主要原因: (1) 覆盖不同设备 (手机、笔记本电脑和边缘服务器) 以及具有不同架构和量化的模型可能表现出不同能耗模式的能耗情况; (2) 针对短对话任务, 在这些任务中, 设备上的大语言模型显示出足够的效率和效果。重要的是, 由于我们的调度和迁移算法都是基于长度阈值的, 只要这些模式是可预测的, 并且在预填充或解码额外 token 时保证消耗更多能量, 它们就能够适应任何复杂的能耗模式。

E.2 FLOPs of On-Device LLMs

To accurately quantify the computational cost per token in both prefill and decode stages, we conduct a detailed FLOPs analysis using three representative models: BLOOM-1.1B, BLOOM-560M, and Qwen1.5-0.5B. All models share a 24-layer architecture but differ in other parameters: BLOOM-1.1B ($d_{\text{model}} = 1024$, 16 heads, FFN dim=4096), BLOOM-560M ($d_{\text{model}} = 512$, 8 heads, FFN dim=2048), and Qwen1.5-0.5B ($d_{\text{model}} = 768$, 12 heads, FFN dim=2048).

Per-token FLOPs computation. The total FLOPs for processing each token consist of five components:

$$\text{FLOPs}_{\text{total}} = \text{FLOPs}_{\text{attn}} + \text{FLOPs}_{\text{ffn}} + \text{FLOPs}_{\text{ln}} + \text{FLOPs}_{\text{emb}} + \text{FLOPs}_{\text{sout}} \quad (9)$$

For a sequence of length L , the attention computation differs between stages. In prefill:

$$\text{FLOPs}_{\text{attn}} = n_{\text{layers}} \cdot \left(3d_{\text{model}}^2 + \frac{L^2 d_{\text{model}}}{n_{\text{heads}}} + Ld_{\text{model}} + d_{\text{model}}^2 \right) \quad (10)$$

While in decode, KV caching eliminates the quadratic term:

$$\text{FLOPs}_{\text{attn}} = n_{\text{layers}} \cdot \left(3d_{\text{model}}^2 + \frac{Ld_{\text{model}}}{n_{\text{heads}}} + Ld_{\text{model}} + d_{\text{model}}^2 \right) \quad (11)$$

Table 6 presents the total FLOPs across different sequence lengths. The decode phase maintains constant FLOPs regardless of sequence length due to KV caching, while prefill phase FLOPs increase with sequence length. A breakdown of computational cost by component (Table 7) reveals that embedding and output projection operations account for the majority of FLOPs, particularly in models with large vocabularies.

E.3 LLM Service Pricing

This section provides further details on the pricing of LLM services. Table 8 presents the pricing models for several commercial Large Language Models (LLMs) as of October 28, 2024. The pricing structure follows a dual-rate model, differentiating between input (prompt) and output (generation) tokens. These rates represent the public pricing tiers available to general users, excluding any enterprise-specific arrangements or volume-based discounts.

Length	BLOOM-1.1B	BLOOM-560M	Qwen-0.5B
<i>Prefill Phase</i>			
L = 32	0.85	0.45	0.39
L = 64	0.93	0.50	0.45
L = 128	1.25	0.65	0.69
<i>Decode Phase</i>			
L = 32	0.82	0.42	0.37
L = 64	0.82	0.42	0.37
L = 128	0.82	0.42	0.37

Table 6: Prefill and Decode FLOPs (billions)

Component	BLOOM-1.1B	BLOOM-560M	Qwen-0.5B
Embedding	31.24	25.00	31.51
Attention	13.01	10.00	16.56
FFN	24.48	20.00	20.38
LayerNorm	0.02	0.02	0.04
Output	31.24	25.00	31.51

Table 7: Component Ratios at L=128 (%)

F Pseudocode for Cost-Aware Adaptive Request Scheduling

The request scheduling algorithm consists of three key components. Algorithm 1 defines the input parameters and determines whether the scenario is device-constrained or server-constrained based on the relative costs. For device-constrained scenarios, Algorithm 2 implements a wait-time strategy to protect tail latency while conserving device energy when possible. For server-constrained scenarios, Algorithm 3 employs a length-based routing approach to optimize TTFT while maintaining the server budget constraint. These algorithms work together to achieve the dual objectives of minimizing latency and managing costs.

Algorithm 1 Variable Definitions and Constraints

Require:

- 1: $p(l)$: Length distribution
- 2: $F(t)$: TTFT CDF of server
- 3: $b \in [0, 1]$: Budget ratio
- 4: c_d^p, c_d^d : Device prefill/decode costs
- 5: c_s^p, c_s^d : Server prefill/decode costs
- 6: $\alpha \in (0, 1)$: Tail ratio

Ensure: Policy type based on cost constraints

- 7: **if** $\min(c_d^p, c_d^d) > \max(c_s^p, c_s^d)$ **then** Device-constrained
- 8: **else** Server-constrained

E.2 设备端大型语言模型的浮点运算量

为了准确量化在预训练和解码阶段每个 token 的计算成本，我们使用三个代表性模型进行了详细的 FLOPs 分析：BLOOM-1.1B、BLOOM-560M 和 Qwen1.5-0.5B。所有模型都采用 24 层架构，但其他参数有所不同：BLOOM-1.1B（ $d_{\text{model}} = 1024$ ，16 个头，FFN 维度 4096）、BLOOM-560M（ $d_{\text{model}} = 512$ ，8 个头，FFN 维度 2048）、Qwen1.5-0.5B（ $d_{\text{model}} = 768$ ，12 个头，FFN 维度 2048）。

每个 token 的 FLOPs 计算。 处理每个

token 的总 FLOPs 包含五个部分：

$$\text{FLOPs}_{\text{total}} = \text{FLOPs}_{\text{attn}} + \text{FLOPs}_{\text{ffn}} + \text{FLOPs}_{\text{ln}} + \text{FLOPs}_{\text{emb}} + \text{FLOPs}_{\text{sout}} \quad (9)$$

对于长度为 L 的序列，注意力计算在不同阶段有所不同。在预训练阶段：

$$\text{FLOPs}_{\text{attn}} = n_{\text{layers}} \cdot \left(3d_{\text{model}}^2 + \frac{L^2 d_{\text{model}}}{n_{\text{heads}}} + Ld_{\text{model}} + d_{\text{model}}^2 \right) \quad (10)$$

在解码时，KV 缓存消除了二次项：

$$\text{FLOPs}_{\text{attn}} = n_{\text{layers}} \cdot \left(3d_{\text{model}}^2 + \frac{Ld_{\text{model}}}{n_{\text{heads}}} + Ld_{\text{model}} + d_{\text{model}}^2 \right) \quad (11)$$

表 6 展示了不同序列长度下的总 FLOPs。由于 KV 缓存，解码阶段的 FLOPs 无论序列长度如何都保持不变，而预填充阶段的 FLOPs 则随着序列长度增加而增加。按组件划分的计算成本（表 7）显示，嵌入和输出投影操作占据了大部分 FLOPs，特别是在词汇量较大的模型中。

E.3 大型语言模型服务定价

本节提供了有关大型语言模型（LLM）服务定价的更多细节。表 8 展示了截至 2024 年 10 月 28 日几种商业大型语言模型（LLM）的定价模式。定价结构采用双费率模型，区分输入（提示）令牌和输出（生成）令牌。这些费率代表面向普通用户的公开定价层，不包括任何企业专属安排或基于使用量的折扣。

Length	BLOOM-1.1B	BLOOM-560M	Qwen-0.5B
<i>Prefill Phase</i>			
L = 32	0.85	0.45	0.39
L = 64	0.93	0.50	0.45
L = 128	1.25	0.65	0.69
<i>Decode Phase</i>			
L = 32	0.82	0.42	0.37
L = 64	0.82	0.42	0.37
L = 128	0.82	0.42	0.37

表 6：预处理和解码 FLOPs（十亿次）

Component	BLOOM-1.1B	BLOOM-560M	Qwen-0.5B
Embedding	31.24	25.00	31.51
Attention	13.01	10.00	16.56
FFN	24.48	20.00	20.38
LayerNorm	0.02	0.02	0.04
Output	31.24	25.00	31.51

表 7：L=128 时的组分比例（%）

F 成本感知自适应请求调度的伪代码

请求调度算法由三个关键组成部分构成。算法 1 定义输入参数，并根据相对成本确定场景是设备受限还是服务器受限。对于设备受限的场景，算法 2 实施等待时间策略，以在可能的情况下保护尾延迟并节省设备能量。对于服务器受限的场景，算法 3 采用基于长度的路由方法，以在保持服务器预算约束的同时优化 TTFT。这些算法共同协作，以实现最小化延迟和管理成本的双重目标。

Algorithm 1 Variable Definitions and Constraints

Require:

- 1: $p(l)$: Length distribution
- 2: $F(t)$: TTFT CDF of server
- 3: $b \in [0, 1]$: Budget ratio
- 4: c_d^p, c_d^d : Device prefill/decode costs
- 5: c_s^p, c_s^d : Server prefill/decode costs
- 6: $\alpha \in (0, 1)$: Tail ratio

Ensure: Policy type based on cost constraints

- 7: **if** $\min(c_d^p, c_d^d) > \max(c_s^p, c_s^d)$ **then** Device-constrained
- 8: **else** Server-constrained

Model	Vendor	Input price	Output price
DeepSeek-V2.5	DeepSeek	0.14	0.28
GPT-4o-mini	OpenAI	0.15	0.60
LLaMa-3.1-70b	Hyperbolic	0.40	0.40
LLaMa-3.1-70b	Amazon	0.99	0.99
Command	Cohere	1.25	2.00
GPT-4o	OpenAI	2.50	10.0
Claude-3.5-Sonnet	Anthropic	3.00	15.0
o1-preview	OpenAI	15.0	60.0

Table 8: LLM service pricing (USD per 1M Tokens). Input prices refer to tokens in the prompt, while output prices apply to generated tokens.

Model	Vendor	Input price	Output price
DeepSeek-V2.5	DeepSeek	0.14	0.28
GPT-4o-mini	OpenAI	0.15	0.60
LLaMa-3.1-70b	Hyperbolic	0.40	0.40
LLaMa-3.1-70b	Amazon	0.99	0.99
Command	Cohere	1.25	2.00
GPT-4o	OpenAI	2.50	10.0
Claude-3.5-Sonnet	Anthropic	3.00	15.0
o1-preview	OpenAI	15.0	60.0

Table 8: LLM service pricing (USD per 1M Tokens). Input prices refer to tokens in the prompt, while output prices apply to generated tokens.

Algorithm 2 Device-constrained Scheduling

Require: Variables from Algorithm 1

```

1: // Phase 1: Set maximum wait time for tail protection
2:  $w_{tail} \leftarrow F^{-1}(1 - \min(\alpha, b))$ 
3: // Initialize wait times for all prompt lengths
4:  $W \leftarrow \{l : w_{tail} \text{ for all } l\}$ 
5: if  $b \leq \alpha$  then
6:   return  $W$  {Use max wait time for all lengths}
7: end if
8: // Phase 2: Optimize wait times with remaining budget
9: available_budget  $\leftarrow b - \alpha$ 
10: for  $l \in \text{sort}(\text{support}(p(l)))$  do
11:   length_cost  $\leftarrow p(l) \cdot l \cdot (1 - \alpha)$ 
12:   if available_budget  $\geq$  length_cost then
13:      $W[l] \leftarrow 0$  {Start device immediately}
14:     available_budget  $\leftarrow$  available_budget - length_cost
15:   else
16:     // Find optimal wait time that meets budget
17:     Find  $w^* \in [0, w_{tail}]$  where:
18:      $F(w^*) \cdot \text{length\_cost} + (b - \text{available\_budget}) = b$ 
19:      $W[l] \leftarrow w^*$ 
20:   break
21: end if
22: end for
23: return  $W$  {Map from prompt lengths to wait times}

```

Algorithm 3 Server-constrained Scheduling

Require: Variables from Algorithm 1

```

1: // Find length threshold to split execution modes
2: Compute  $l_{th}$  where:  $\int_0^{l_{th}} l \cdot p(l) dl = (1 - b) \cdot \int_0^\infty l \cdot p(l) dl$ 
3: // Initialize execution policy map
4:  $P \leftarrow \emptyset$ 
5: for  $l \in \text{support}(p(l))$  do
6:   if  $l < l_{th}$  then
7:      $P[l] \leftarrow (1, 0)$   $\{(I_d, I_s): \text{Device only}\}$ 
8:   else
9:      $P[l] \leftarrow (1, 1)$   $\{(I_d, I_s): \text{Concurrent execution}\}$ 
10:  end if
11: end for
12: return  $P$  {Map from lengths to execution indicators}

```

Algorithm 2 Device-constrained Scheduling

Require: Variables from Algorithm 1

```

1: // Phase 1: Set maximum wait time for tail protection
2:  $w_{tail} \leftarrow F^{-1}(1 - \min(\alpha, b))$ 
3: // Initialize wait times for all prompt lengths
4:  $W \leftarrow \{l : w_{tail} \text{ for all } l\}$ 
5: if  $b \leq \alpha$  then
6:   return  $W$  {Use max wait time for all lengths}
7: end if
8: // Phase 2: Optimize wait times with remaining budget
9: available_budget  $\leftarrow b - \alpha$ 
10: for  $l \in \text{sort}(\text{support}(p(l)))$  do
11:   length_cost  $\leftarrow p(l) \cdot l \cdot (1 - \alpha)$ 
12:   if available_budget  $\geq$  length_cost then
13:      $W[l] \leftarrow 0$  {Start device immediately}
14:     available_budget  $\leftarrow$  available_budget - length_cost
15:   else
16:     // Find optimal wait time that meets budget
17:     Find  $w^* \in [0, w_{tail}]$  where:
18:      $F(w^*) \cdot \text{length\_cost} + (b - \text{available\_budget}) = b$ 
19:      $W[l] \leftarrow w^*$ 
20:   break
21: end if
22: end for
23: return  $W$  {Map from prompt lengths to wait times}

```

Algorithm 3 Server-constrained Scheduling

Require: Variables from Algorithm 1

```

1: // Find length threshold to split execution modes
2: Compute  $l_{th}$  where:  $\int_0^{l_{th}} l \cdot p(l) dl = (1 - b) \cdot \int_0^\infty l \cdot p(l) dl$ 
3: // Initialize execution policy map
4:  $P \leftarrow \emptyset$ 
5: for  $l \in \text{support}(p(l))$  do
6:   if  $l < l_{th}$  then
7:      $P[l] \leftarrow (1, 0)$   $\{(I_d, I_s): \text{Device only}\}$ 
8:   else
9:      $P[l] \leftarrow (1, 1)$   $\{(I_d, I_s): \text{Concurrent execution}\}$ 
10:  end if
11: end for
12: return  $P$  {Map from lengths to execution indicators}

```